POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C for disciplined convex programming

WILLIAM ZHANG
Département de Génie Électrique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Génie Électrique

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C
for disciplined convex programming

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Sébastien LE DIGABEL, président
Antoine LESAGE-LANDRY, membre et directeur de recherche
Antoine LEGRAIN, membre

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

I gratefully acknowledge the Natural Sciences and Engineering Research Council of Canada (NSERC) for funding this research, and [source] for the scholarship that supported my stay at Stanford.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xiii

CHAPTER 1 INTRODUCTION 1
1.1 Problem statement 2
1.2 Canonicalization 4
1.2.1 Conic reformulation 6
1.2.2 `CVXPY` 7
1.2.3 Example 7
1.3 Computing derivatives 10
1.3.1 Numerical differentiation 10
1.3.2 Symbolic differentiation 11
1.3.3 Algorithmic differentiation 11
1.4 Matrix computations 12
1.4.1 Dense matrices 13
1.4.2 Sparse matrices 14
1.4.3 Sparse matrix products 15
1.5 Contributions 16

CHAPTER 2 LITERATURE REVIEW 17
2.1 Matrix stuffing in `CVXPY` 17
2.1.1 Composing linear operators 18
2.1.2 Example 19
2.1.3 Parameter caching 20
2.2 Sparse derivatives from detected structure 23
2.2.1 Matrix-vector products and accumulation 23

2.2.2 Compression by graph colouring . . . 25
2.2.3 `CasADi` . . . 27
2.3 Coefficient extraction from declared structure . . . 28
2.3.1 `AMPL` . . . 28
2.3.2 `JuMP` . . . 29

CHAPTER 3 A LIGHTWEIGHT CANONICALIZATION ENGINE . . . 32
3.1 Overview of the engine . . . 32
3.1.1 Atom classes . . . 33
3.1.2 The expression node . . . 33
3.2 Matrix representations . . . 34
3.2.1 The permuted dense matrix . . . 35
3.2.2 The stacked permuted dense matrix . . . 37
3.3 Computational kernels . . . 39
3.3.1 Kernel dispatch . . . 39
3.3.2 Reusing work across passes . . . 40
3.4 The API and the CVXPY interface . . . 41

CHAPTER 4 BENCHMARK RESULTS . . . 43
4.1 Benchmark Setup . . . 43
4.2 Single-solve compilation against `cvxcore` . . . 44
4.3 Single-solve compilation against the `SCIPY` and `COO` backends . . . 44
4.4 Parametric compilation against `cvxcore` . . . 47

CHAPTER 5 CONCLUSION . . . 49
5.1 Summary of Works . . . 49
5.2 Limitations . . . 49
5.3 Future Research . . . 49

REFERENCES . . . 50

LIST OF TABLES

Table 1.1	A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output.	4
Table 2.1	A subset of the linear operators, classified as views or copies	19
Table 2.2	The reviewed systems and the engine of Chapter 3, sorted by the origin of their problem-data coefficients: stored as the user typed them, detected at runtime, or differentiated out of an expression graph. . .	31
Table 3.1	The atom classes and how each computes its Jacobian.	33
Table 4.1	Cold compilation time, `diffengine` versus `cvxcore`	45
Table 4.2	Cold compilation time under all three `CVXPY` backends	46
Table 4.3	Parametric compilation, `diffengine` versus `cvxcore`	48

LIST OF FIGURES

Figure 1.1 Example of an affine expression tree for $\mathbf{Ax} - \mathbf{b}$. Variables are drawn as circles and constants (or parameters) as boxes. 5

Figure 1.2 Before canonicalization. `CVXPY` represents the DCP problem (1.5) as these expression trees. Variables are labelled with a circle, and parameters with a square. 8

Figure 1.3 After canonicalization. `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $\mathbf{x}$ appears as a leaf of several trees; all occurrences refer to the same variable. 9

Figure 1.4 The least squares objective $f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{2\times 2}$ as a scalar and as a vectorized computational graph. In the scalar graph every edge carries a scalar partial derivative. In the vectorized graph the local derivative of each node is a Jacobian block, shown below the node: $\mathbf{A}$ for the affine node and $2\mathbf{r}^\top$ for the sum of squares, where $\mathbf{r} = \mathbf{Ax} - \mathbf{b}$. 12

Figure 1.5 Dense storage of a 2×3 matrix: the same entries as one contiguous array in row-major and in column-major order. 13

Figure 1.6 The product $\mathbf{y} = \mathbf{Sv}$ with the matrix of (1.10). In CSR, the nonzeros of one row form a dot product that produces one output entry. In CSC, the nonzeros of one column are scaled by one input entry and accumulated into several output entries. 15

Figure 2.1 Jacobians under the vectorization (2.1), for a matrix variable $\mathbf{X}$ and the product $\mathbf{FX}$, with $\mathbf{X}, \mathbf{F} \in \mathbb{R}^{2\times 2}$. The entries of $\mathbf{vec}(\mathbf{X})$, in column-major order, index the columns of the Jacobians. 18

Figure 2.2 The column offsets of a stack of three variables $\mathbf{z} = (\mathbf{u}, \mathbf{v}, \mathbf{w})$ with $\mathbf{u} \in \mathbb{R}^2$, $\mathbf{v} \in \mathbb{R}^3$, and $\mathbf{w} \in \mathbb{R}^2$. Each offset is the total dimension of the preceding variables: $\mathbf{u}$ starts at column 0, $\mathbf{v}$ at 2, and $\mathbf{w}$ at 5. The entries of $\mathbf{z}$ index the columns of every augmented matrix $[\mathbf{A}_f \ \mathbf{b}_f]$. . . 20

Figure 2.3 Matrix stuffing of problem (1.5) by a bottom-up pass over the conic-form expression trees of Figure 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children. The coefficients appear negated because (1.2) writes the constraints as $\mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}$. The horizontal rules in $[\mathbf{A}\ \mathbf{b}]$ separate the rows of the three constraints. 21

Figure 2.4 What each layout stores during the pass, at the node $-(\mathbf{F}\mathbf{x} - \mathbf{g})$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns. 21

Figure 2.5 The DPP path on the objective data of problem (1.5): the parameter map $\mathbf{Q}$ is built and cached once, and each re-solve recovers $\mathbf{c}$ with the single sparse product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$. 22

Figure 2.6 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the pass is repeated on the parameter-free tree on every change of the parameters. 22

Figure 2.7 Forward sweep of a computational graph 24

Figure 2.8 Reverse sweep of a computational graph 24

Figure 2.9 Accumulation by vertex elimination 25

Figure 2.10 Graph colouring compresses the seeds 26

Figure 3.1 A permuted dense matrix. The dense block $\mathbf{X}$ is stored once, contiguously; the index vectors $\mathbf{r}$ and $\mathbf{c}$ place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored. Each block row keeps its own shade, reused in Figure 3.2. 35

Figure 3.2 Row indexing of the permuted dense matrix $\mathbf{A}$ of Figure 3.1. The table traces the scan of the inverse row map: each requested global row yields a block row of $\mathbf{X}$ to copy, or -1, in which case the output row is structurally zero and not stored. Shades follow the block rows of Figure 3.1. The result is the permuted dense triple $(\mathbf{X}_C, \mathbf{r}_C, \mathbf{c}_C)$. . 36

Figure 3.3 The Jacobian of $\mathbf{A}\mathbf{X}$ as a stacked permuted dense matrix, for dense $\mathbf{A} \in \mathbb{R}^{2\times 2}$ and a variable $\mathbf{X} \in \mathbb{R}^{2\times 2}$ in a problem with six variables. There is one block per column of $\mathbf{X}$, each a permuted dense copy of $\mathbf{A}$ at its own rows and columns. The 4×6 matrix $\mathbf{M}$ is never assembled; a product with $\mathbf{M}$ is one `dgemm` per block. 38

Figure 3.4 Double dispatch of the chain-rule products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{D}\mathbf{A}$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its colour indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel. 39

Figure 3.5 The two passes of the `sum` atom. During the symbolic pass, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The numeric passes of the other structural atoms have the same form. 41

LIST OF SYMBOLS AND ACRONYMS

AD	Algorithmic Differentiation
AML	Algebraic Modelling Language
AMPL	A Mathematical Programming Language
API	Application Programming Interface
ASA	Affine-Solver-Affine
ASL	AMPL Solver Library
BLAS	Basic Linear Algebra Subprograms
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
JVP	Jacobian-Vector Product
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SIMD	Single Instruction, Multiple Data
SOCP	Second-Order Cone Program
SDP	Semidefinite Program
VJP	Vector-Jacobian Product

CHAPTER 1 INTRODUCTION

Since the early days of linear programming, optimization practitioners have needed to translate mathematical formulations into data that solvers can process. At first, this meant encoding each coefficient of the problem matrices by hand [1]. Matrix generators were introduced in the late 1960s to automate this task: they compute the coefficients through explicit loops and write them out in solver-readable formats like MPS [2, 3]. Matrix generators were able to remove most of the manual labour, but a gap still remained. Modellers naturally think in terms of sets, indices, and algebraic expressions, while procedural code constructs the coefficient matrices entry by entry. This mismatch between how a model is expressed and how it is implemented made models difficult to verify, modify, and communicate with others [4]. Algebraic modelling languages such as `AMPL` [5], `AIMMS` [6], and `GAMS` [7] closed this gap by accepting problem descriptions that closely follow the mathematical formulation. The survey [8] provides a complete overview of the early history of modelling languages.

More recently, modelling languages embedded in general-purpose programming languages have gained popularity. These include `Pyomo` [9,10] in Python, `JuMP` [11] in Julia, `CasADi` [12] in C++, and `YALMIP` [13] in MATLAB. Other domain-specific modelling languages specialized for convex optimization [14] are built on disciplined convex programming (DCP), the composition rules under which the convexity of a problem can be verified automatically [15, 16]. These DCP-based languages include `CVX` [17] in MATLAB, `CVXR` [18] in R, `CVXPY` [19] in Python, and `Convex.jl` [20] in Julia.

In the meantime, modern general-purpose solvers [21–23] have mostly transitioned towards conic programming, which generalizes linear programming and can represent and solve nearly every convex program used in practice [24]. The translation task remains a responsibility of the modelling languages, which now target conic problem data instead of Linear Program (LP) coefficient matrices. In DCP-based languages, this translation process is called canonicalization [25]. While the time taken to solve an optimization problem is often dominated by the solver, the time taken to canonicalize (or *compile*) it can also be significant [26]. The main goal of this Master's thesis is to reduce the compilation time of `CVXPY`.

1.1 Problem statement

The standard form of a convex optimization problem is

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & f_0(\mathbf{x};\boldsymbol{\theta}) \\ \text{subject to} & f_i(\mathbf{x};\boldsymbol{\theta}) \le 0, \quad i = 1,\dots,m_1 \\ & g_j(\mathbf{x};\boldsymbol{\theta}) = 0, \quad j = 1,\dots,m_2 \end{array} \tag{1.1}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the optimization variable, the objective $f_0 : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ and inequality constraint functions $f_1, \dots, f_{m_1} : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ are convex, and the equality constraint functions $g_1, \dots, g_{m_2} : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ are affine [14]. The parameter $\boldsymbol{\theta} \in \mathbb{R}^p$ specifies data that can change between solves but is fixed and given when solving an instance of the problem. We call the parametrized problem (1.1) a problem family; for a fixed value of $\boldsymbol{\theta}$, we call it a problem instance.

Canonicalization converts the high-level description (1.1) to a quadratic conic form used by many general-purpose convex solvers

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \frac{1}{2}\mathbf{x}^\top \mathbf{P}(\boldsymbol{\theta})\mathbf{x} + \mathbf{c}(\boldsymbol{\theta})^\top \mathbf{x} \\ \text{subject to} & \mathbf{b}(\boldsymbol{\theta}) - \mathbf{A}(\boldsymbol{\theta})\mathbf{x} \in \mathcal{K} \end{array} \tag{1.2}$$

where the *problem data* are the symmetric positive semidefinite matrix $\mathbf{P}(\boldsymbol{\theta}) \in \mathbb{S}^n_+$, the matrix $\mathbf{A}(\boldsymbol{\theta}) \in \mathbb{R}^{m\times n}$, the vectors $\mathbf{b}(\boldsymbol{\theta}) \in \mathbb{R}^m$ and $\mathbf{c}(\boldsymbol{\theta}) \in \mathbb{R}^n$, and the non-empty closed convex cone $\mathcal{K} \subseteq \mathbb{R}^m$. Problem (1.2) generalizes both quadratic programs (QPs), when $\mathcal{K} = \mathbb{R}^m_+$ is the nonnegative orthant, and standard conic programs, when $\mathbf{P}(\boldsymbol{\theta}) = 0$.

Most of the cost of canonicalization lies in the final step, where symbolic expression trees are converted to numerical problem data. This step is often called matrix stuffing or the backend of canonicalization [27]. During the first solve of a problem family, `CVXPY` forms and caches a parameter tensor that maps the parameter vector to the problem data. On subsequent solves, new problem data can be obtained with a single matrix-vector product [26].

For large-scale problems, forming the parameter tensor can be prohibitively expensive, so `CVXPY` lets users opt out of it entirely. Users must thus choose between paying a large upfront cost on the first solve and repeating the full canonicalization, with nothing reused, at every solve. We review `CVXPY` and the frontend of canonicalization in §1.2, as well as the parameter tensor and matrix stuffing in §2.1.

The constraint matrix $\mathbf{A}$ is the Jacobian of the stacked constraint expressions and $\mathbf{P}$ is the Hessian of the objective, so matrix stuffing amounts to computing a sparse Jacobian and

Hessian. Classical methods build these matrices column by column with multiple passes of algorithmic differentiation (AD), using graph colouring to group columns that can be recovered in a single pass [28, 29]. These methods treat the expression as a blackbox: the sparsity pattern must first be detected and the columns grouped before a single entry of the matrix is obtained. We review the basics of computing derivatives in §1.3 and present sparse AD and graph colouring methods in §2.2.

A modelling language, by contrast, knows which variables enter each constraint. `AMPL` (A Mathematical Programming Language) is split into two components: its commercial translator writes the structure of the model to a `.nl` file [30], and the open source AMPL Solver Library (ASL) evaluates constraints and their derivatives from this file inside the solver process [31]. `Pyomo` also interfaces to solvers through `.nl` files, among other formats, though it generally writes them more slowly than the `AMPL` translator.

On the other hand, `JuMP` has no single standard form to target; it decouples modeller and solver through MathOptInterface (MOI), a typed in-memory interface [32]. The model is built constraint by constraint, and each constraint stores its coefficients directly, so extraction reduces to copying them into the format of the chosen solver. None of these approaches operates on expression trees: the AD tools record the expression as a flat tape of instructions, `AMPL` exchanges the model through a file, and `JuMP` stores lists of coefficients. In `CVXPY` the expression trees themselves are the input of the backend. We review both these modelling languages in §2.3.

Our approach. We make use of the fact that the sparsity patterns of $\mathbf{A}$ and $\mathbf{P}$ are fixed across a problem family, because the parameter $\boldsymbol{\theta}$ changes the values of the problem data but not their structure. We can therefore split matrix stuffing into a symbolic and a numeric phase. The symbolic phase runs once on the first solve, where it computes the sparsity patterns, allocates memory for all intermediate matrices, and caches other computations that remain valid across the problem family. The numeric phase then fills in new values at every solve. This separation between structure and values is inspired by the interfaces of Nonlinear Program (NLP) solvers such as `IPOPT` [33, 34].

The symbolic phase is much cheaper to form and cache than the parameter tensor, and the numeric phase is often similar in cost to applying the matrix-vector product. The problem data is also assembled entirely in memory rather than through a separate model file. Finally, we exploit structure within the sparsity patterns themselves, which often contain dense blocks of data. When possible, we store these blocks as dense matrices and compute with dense linear algebra kernels, rather than always relying on generic sparse formats and routines (we

provide background on dense and sparse matrix computations in §1.4).

1.2 Canonicalization

The problem family (1.1) is written in functional form, as objective and constraint functions of the variable and the parameters. In a modelling language, the optimization variables $\mathbf{x}_1, \ldots, \mathbf{x}_k$ are collected into a single vector $\mathbf{x} \in \mathbb{R}^n$, and likewise the parameters $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_l$ into $\boldsymbol{\theta} \in \mathbb{R}^p$ [14]. Users are not required to write constraints with a zero right-hand side as in (1.1). Modelling languages accept inequalities with a convex left-hand side and a concave right-hand side, and equalities with both sides affine. Converting to the form (1.1) is trivial and only requires moving every term to the left-hand side. Internally, each function is represented as an *expression tree*.

Modelling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in `CVXPY` because they are the fundamental building blocks of the modelling language [15]. Alongside its numerical definition, each atom declares useful attributes such as its curvature, monotonicity with respect to each argument, and the sign of its output. A selection of common atoms is shown in Table 1.1.

Table 1.1 A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output.

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`exp`	e^x	convex	non-decreasing	positive
`log`	$\log x$	concave	non-decreasing	any
`norm`	$\lVert x \rVert_2$	convex	unknown	nonnegative
`max`	$\max_i x_i$	convex	non-decreasing	any
`sum_largest`	sum of k largest entries	convex	non-decreasing	any

An expression is defined recursively as an atom evaluated at *subexpressions*, each of which is itself another expression. Such an expression is naturally stored as an expression tree, with atoms at its internal nodes and variables, parameters, and constants at its leaves. In Python, this tree can be formed automatically from user code through operator overloading. Most operations on an expression, from evaluation to verifying convexity to differentiation,

traverse the tree and combine the intermediate results at each node.

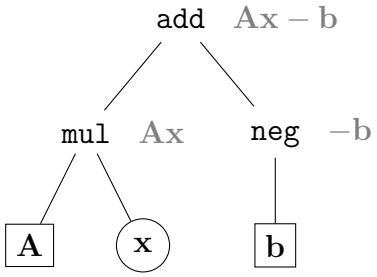


Figure 1.1 Example of an affine expression tree for $\mathbf{Ax} - \mathbf{b}$. Variables are drawn as circles and constants (or parameters) as boxes.

To certify the convexity of a problem, DCP-based modelling languages use the DCP composition rules to classify the curvature of each function [15]. Once all functions are classified, a problem is *DCP-compliant* if the objective and each constraint have curvatures compatible with a convex problem: a convex objective, convex $\leq$ concave inequalities, and affine equalities. Otherwise, if a curvature is incompatible (*e.g.*, f_0 is concave) or cannot be determined, the modelling language raises an error and terminates. Because these rules are only sufficient for convexity, a problem that is genuinely convex but not expressed in a recognized form may still be rejected.

The composition $\phi(u_1(\mathbf{x}), u_2(\mathbf{x}), \ldots, u_k(\mathbf{x}))$ of an atom $\phi : \mathbb{R}^k \to \mathbb{R}$ with subexpressions $u_i : \mathbb{R}^n \to \mathbb{R}$ is convex when ϕ is convex and, for each argument $i \in \{1, \ldots, k\}$, one of the following holds:

- ϕ is non-decreasing in argument i and u_i is convex;
- ϕ is non-increasing in argument i and u_i is concave; or
- u_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require ϕ and all of its arguments to be affine [15].

1.2.1 Conic reformulation

Once a problem is verified to be DCP-compliant, canonicalization rewrites its functional form (1.1) into an equivalent functional conic form

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & f_0(\mathbf{x};\boldsymbol{\theta}) \\ \text{subject to} & h_i(\mathbf{x};\boldsymbol{\theta}) \in \mathcal{K}_i, \quad i = 1,\dots,m, \end{array} \tag{1.3}$$

where f_0 is affine or quadratic, each $h_i : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^{d_i}$ is affine, and each $\mathcal{K}_i \subseteq \mathbb{R}^{d_i}$ is one of the standard cones supported by general-purpose conic solvers: the zero, nonnegative, second-order, power, exponential, and positive semidefinite cones.

Every nonlinear atom is rewritten into conic constraints in three steps. The first step rewrites the problem into *Smith form*, where each nonlinear atom is replaced by a new variable coupled with an equality constraint; every atom then acts only on affine arguments [35]. For a convex atom ϕ, the Smith form gives an auxiliary variable $t \in \mathbb{R}$ with the equality constraint $\phi(\mathbf{x}) = t$. The second step relaxes this equality to the inequality $\phi(\mathbf{x}) \le t$. The DCP rules allow a convex atom only where its value is minimized or upper-bounded, so the solver always prefers t as small as possible; the relaxation is therefore lossless. The pair $(\mathbf{x}, t)$ then lies in the *epigraph* $\{(\mathbf{x}, t) \in \mathbb{R}^n \times \mathbb{R} : \phi(\mathbf{x}) \le t\}$ of ϕ, so we often refer to t as an *epigraph variable*. A concave atom relaxes to $\phi(\mathbf{x}) \ge t$ instead, giving a *hypograph variable*. The third and final step expresses each epigraph (hypograph) inequality as the membership of an affine function of $(\mathbf{x}, t)$ in a convex cone [36].

Example of reformulations. A first example is the ℓ_2 norm atom $\|\mathbf{x}\|_2$ of $\mathbf{x} \in \mathbb{R}^n$, whose epigraph $\|\mathbf{x}\|_2 \le t$ is exactly the second-order cone constraint $(t, \mathbf{x}) \in \mathcal{K}_{\text{soc}}$, where $\mathcal{K}_{\text{soc}} = \{(t, \mathbf{x}) \in \mathbb{R} \times \mathbb{R}^n : \|\mathbf{x}\|_2 \le t\}$. A second example is the nondifferentiable absolute value atom $|x|$ of a scalar x, whose epigraph $|x| \le t$ can be written as the pair of affine inequalities $-t \le x \le t$. A final example is the sum of the k largest entries of $\mathbf{x}$, which expands into a small LP: the epigraph sum_largest$(\mathbf{x}, k) \le t$ holds if and only if there exist a scalar $q \in \mathbb{R}$ and a vector $\mathbf{w} \in \mathbb{R}^n$ such that

$$q\mathbf{1} + \mathbf{w} \ge \mathbf{x}, \qquad \mathbf{w} \ge 0, \qquad kq + \mathbf{1}^\top \mathbf{w} \le t, \tag{1.4}$$

where the inequalities are applied element-wise. Here q acts as a threshold and each w_i measures how far the entry x_i rises above it, so that the sum of the k largest entries is bounded by $kq + \mathbf{1}^\top \mathbf{w}$ [14].

1.2.2 `CVXPY`

`CVXPY` is a DCP-based modelling language embedded in Python [19], which began in 2013 as an implementation of the modelling principles of `CVX` [17]. It differed from `CVX` in three design decisions: modular components built around a few shared data structures, the expression tree formed by Python's operator overloading as the central representation, and support for parameters, which lets compilation work be reused when a problem is solved repeatedly. These ideas matured with version 1.0, which redesigned canonicalization as a general reduction-based rewriting system [25]. The new design enabled the addition of other disciplined grammars [34,37,38], and the early parameter support grew into DPP, a restricted grammar under which canonicalized programs can be updated efficiently [26].

The open source project has since grown through contributions from many other enthusiastic developers; a recent paper surveys the new canonicalization and modelling features introduced between versions 1.1 and 1.9 [27].

The rewriting from the functional form (1.1) to the functional conic form (1.3) is carried out as a sequence of *reductions*. Each reduction provides an `apply` step that converts a problem into an equivalent one, and a `retrieve` step that recovers a solution of the original problem from a solution of the rewritten one [25]. A first example is `FlipObjective`, which turns a maximization problem into a minimization one by flipping the sign of the objective, and flips it back in its `retrieve` step. A second example is `CvxAttr2Constr`, which replaces variable attributes such as nonnegativity with explicit constraints when the target solver does not support them natively.

The main reduction, `Dcp2Cone`, applies the epigraph reformulation of §1.2.1 in a bottom-up pass over the expression trees. The problem is first rewritten into Smith form, each resulting equality is relaxed to an epigraph (or hypograph) inequality, and each inequality is finally expressed as a cone membership [35, 36]. In practice the three steps are performed at once: each nonlinear atom is replaced directly by its epigraph variable and cone constraints, and the intermediate Smith form and its relaxation never appear explicitly.

1.2.3 Example

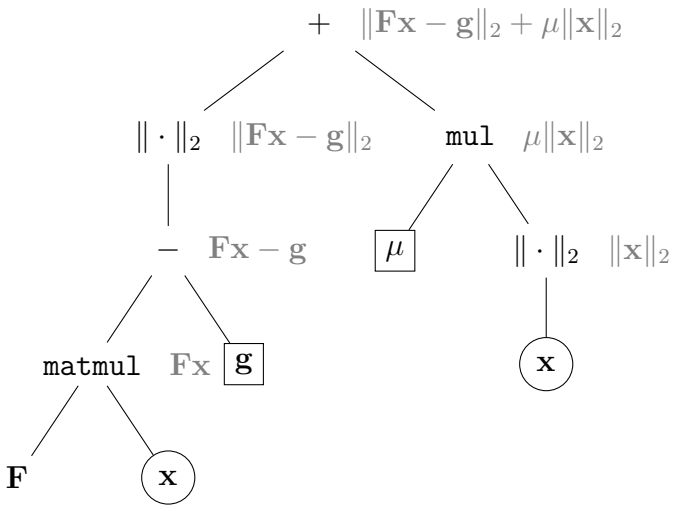
We consider the following simple fitting with regularization problem

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \|\mathbf{F}\mathbf{x}-\mathbf{g}\|_2+\mu\|\mathbf{x}\|_2 \\ \text{subject to} & \mathbf{x}\geq 0 \end{array} \tag{1.5}$$

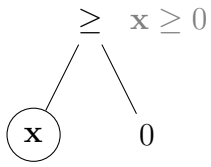
with variable $\mathbf{x} \in \mathbb{R}^n$, fixed problem data $\mathbf{F} \in \mathbb{R}^{m \times n}$, and parameters $\mathbf{g} \in \mathbb{R}^m$ and $\mu \geq 0$; the constraint $\mathbf{x} \geq 0$ requires every entry of $\mathbf{x}$ to be nonnegative. The problem formulation in `CVXPY` code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library `NumPy` [39] for the data arrays. The expression tree representation of the problem is shown in Figure 1.2.



(a) Expression tree of the objective



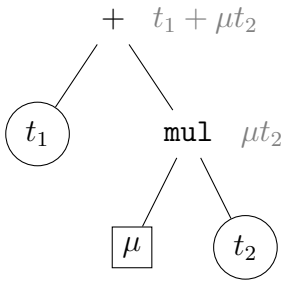
(b) Expression tree of the constraints

Figure 1.2 Before canonicalization. `CVXPY` represents the DCP problem (1.5) as these expression trees. Variables are labelled with a circle, and parameters with a square.

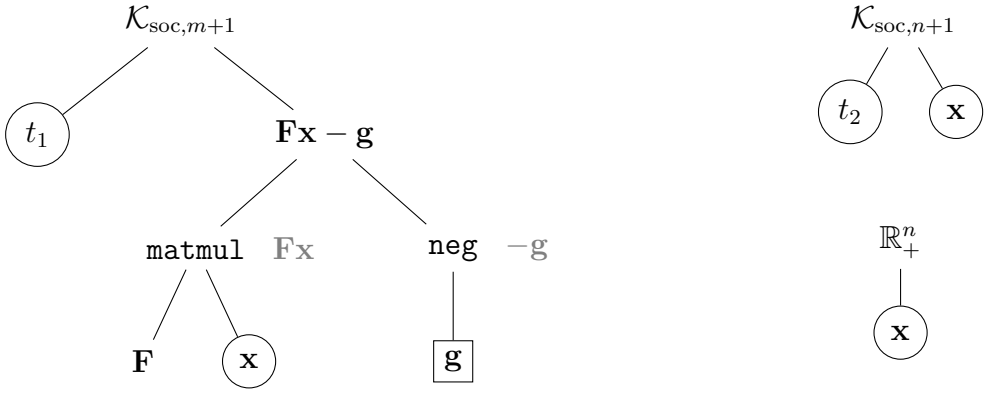
After applying the reductions, the problem has been *canonicalized* to

$$\begin{array}{ll} \underset{t_1, t_2, \mathbf{x}}{\text{minimize}} & t_1 + \mu t_2 \\ \text{subject to} & \mathbf{x} \geq 0 \\ & (t_1, \mathbf{Fx} - \mathbf{g}) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, \mathbf{x}) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{1.6}$$

where $t_1, t_2 \in \mathbb{R}$ are new epigraph variables, and the cone $\mathcal{K}$ from (1.2) is the product $\mathbb{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. Problem (1.6) is now in the functional conic form of §1.2.1: every constraint is a cone membership, and the objective and constraint expressions are affine. Comparing the DCP trees of Figure 1.2 with the trees of Figure 1.3 shows the effect of canonicalization: the nonlinear norm atoms have been replaced by epigraph variables, leaving affine expression trees in the variables, which we call the conic-form trees of the problem. The three rows of (1.6) remain three separate cone constraints, each carrying the affine expressions that must lie in its cone; Figure 1.3 accordingly shows one expression tree per constraint, rather than a single combined tree.



(a) Expression tree of the objective



(b) One expression tree per constraint

Figure 1.3 After canonicalization. `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $\mathbf{x}$ appears as a leaf of several trees; all occurrences refer to the same variable.

1.3 Computing derivatives

Computing derivatives is a fundamental task in scientific computing, with many methods that differ in cost, in accuracy, and in the form of the result [40]. The derivatives in this Master's thesis are the constraint Jacobian $\mathbf{A}$ and the objective Hessian $\mathbf{P}$ of a canonicalized problem, both large sparse matrices. This section presents the Jacobian and the Hessian, the elementary methods for computing them, and the choices that distinguish the software implementations; §2.2 reviews sparse AD and graph colouring methods.

For a vector-valued map $F : \mathbb{R}^n \to \mathbb{R}^m$ of an input $\mathbf{x} \in \mathbb{R}^n$, the first-order derivative is the Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ of partial derivatives of the outputs with respect to the inputs,

$$J_{ij} = \frac{\partial F_i}{\partial x_j}, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n. \tag{1.7}$$

We mainly consider the constraint function, which stacks the potentially many affine expressions of a canonicalized problem into a single vector function of the variables. Its Jacobian is the constant constraint matrix $\mathbf{A}$ of (1.2), whose sparsity pattern is fixed although its values may change from one solve to the next.

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the second-order derivative is the Hessian $\mathbf{H} \in \mathbb{R}^{n \times n}$ of second partial derivatives,

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}, \qquad i, j = 1, \ldots, n. \tag{1.8}$$

The Hessian is symmetric whenever f is twice continuously differentiable, and it is then the Jacobian of the gradient of f. It can therefore be obtained by differentiating the gradient with the same methods used for the Jacobian, although dedicated methods that compute it directly also exist. The function we consider is the canonicalized objective, whose Hessian is the matrix $\mathbf{P}$ of (1.2).

1.3.1 Numerical differentiation

Other than computing derivatives analytically, the simplest method is to use numerical differentiation, which approximates the derivative of a function at a point by evaluating the function at nearby points using a finite difference formula such as

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}, \tag{1.9}$$

where h is a small step size. Note that the central variant $f'(x) \approx (f(x+h) - f(x-h))/2h$ improves accuracy. Both formulas use two function evaluations, but the forward difference can reuse the already computed value $f(x)$ across directions, while the central variant cannot. The approach is simple but has two downsides. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to truncation errors, when h is too large, or rounding errors, when h is too small, in the computed derivative if the step size is not carefully chosen [41]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations, also known as a derivative checker [42].

1.3.2 Symbolic differentiation

Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This symbolic approach to differentiation is used in computer algebra systems such as `Mathematica` [43] and `Maple` [44] as well as the `SymPy` library in Python [45]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can quickly form exponentially large symbolic expressions, which is impractical and inefficient for many problems [46]. When intermediate expressions are stored as common subexpressions, and the expression is represented with a directed acyclic graph (DAG) instead of a tree, symbolic differentiation performs the same set of operations as reverse mode differentiation [47].

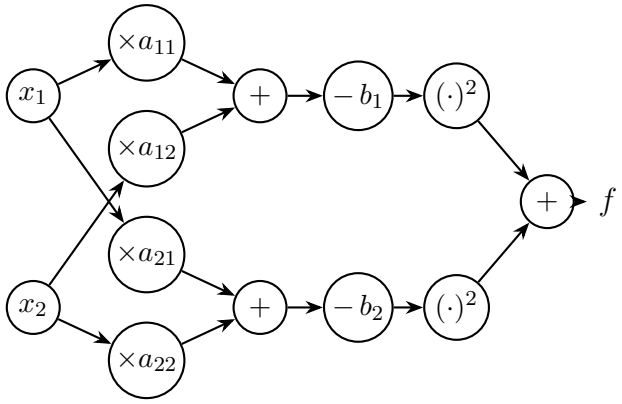
1.3.3 Algorithmic differentiation

Algorithmic differentiation (AD) treats the function as a computational graph of elementary operations, each with a known local derivative rule. The chain rule is then applied to combine these local derivatives throughout the graph, giving the derivative of the whole function. AD is also commonly referred to as automatic differentiation. We prefer the term algorithmic to emphasize that AD differentiates the algorithm that computes a function, rather than a numerical approximation or a symbolic formula [46].

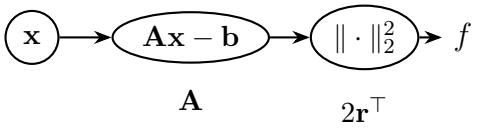
In this broad sense, AD can refer to any method that applies the chain rule over a compu-

tational graph. Its implementations differ in four basic choices: whether the graph is scalar or vectorized, whether the chain rule runs forward or in reverse, whether the derivative is assembled directly or accessed through matrix–vector products, and whether the result is stored sparse or dense.

In most AD tools, every node is a single elementary operation, an addition, a multiplication, or a univariate function, and every edge carries a scalar partial derivative [46]. In a vectorized graph, a node is a whole vector or matrix operation, and its local derivative is a matrix, a block of the Jacobian. Figure 1.4 contrasts the two on a small least squares objective. `CasADi` [12] offers both, through its scalar SX and vectorized MX expressions.



(a) Scalar graph: one node per elementary operation



(b) Vectorized graph: one node per matrix operation

Figure 1.4 The least squares objective $f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{2\times 2}$ as a scalar and as a vectorized computational graph. In the scalar graph every edge carries a scalar partial derivative. In the vectorized graph the local derivative of each node is a Jacobian block, shown below the node: $\mathbf{A}$ for the affine node and $2\mathbf{r}^\top$ for the sum of squares, where $\mathbf{r} = \mathbf{Ax} - \mathbf{b}$.

The modes of AD differ in the order in which the local derivatives are accumulated. Forward mode (or tangent mode) propagates derivatives from the inputs toward the outputs, following the order of evaluation. Reverse mode (or adjoint mode) propagates them from the outputs back toward the inputs, and requires storing the intermediate values of the evaluation [40]. In forward mode the work scales with the number of input variables, and in reverse mode with the number of output variables.

1.4 Matrix computations

The derivatives of §1.3 are matrices, and every matrix is stored in either a dense representation, which keeps all entries in a single array, or a sparse representation, which keeps only the nonzero entries alongside their positions. The representation determines the memory a

matrix occupies and the algorithms that operate on it, and each has its own use cases. This section reviews the storage formats, algorithms, and some of the history behind both types of matrices.

1.4.1 Dense matrices

A dense matrix is stored as a single contiguous array of its entries, ordered row by row (row-major) or column by column (column-major), as shown in Figure 1.5. The position of an entry is computed directly from its indices, so no index arrays are needed, and rows or columns are traversed sequentially in memory. C arrays are row-major and Fortran arrays are column-major. Most dense linear algebra interfaces follow the Fortran convention.

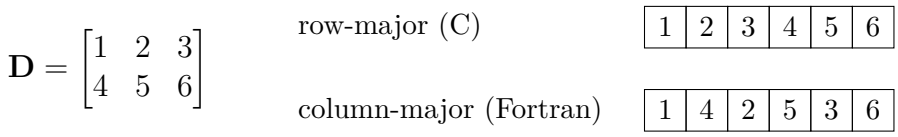


Figure 1.5 Dense storage of a 2×3 matrix: the same entries as one contiguous array in row-major and in column-major order.

Dense linear algebra is standardized by the Basic Linear Algebra Subprograms (BLAS), an interface of vector and matrix kernels [48, 49]. The BLAS is a specification rather than an implementation. Implementations such as `OpenBLAS`, Intel `MKL`, and Apple `Accelerate` are specialized for their respective processors, and nearly every machine has one available, whether shipped with the operating system or bundled by numerical libraries. LAPACK, the library behind dense factorizations and linear system solving in most scientific software, is built on top of the BLAS kernels [50].

The BLAS interface is organized in three levels by the ratio of computation to data. Level 1 covers vector operations such as dot products, scaled copies, and additions, level 2 covers matrix–vector products, and level 3 covers matrix–matrix products. The routine names abbreviate the precision and the operation. For example, `dgemv` is the double-precision general matrix–vector product, and `dgemm` is the double-precision general matrix–matrix product.

BLAS implementations are fast because they are organized around the memory hierarchy. A modern processor multiplies numbers far faster than it loads them from main memory, so optimized kernels reuse data already in cache and process several entries per instruction with Single Instruction, Multiple Data (SIMD) units [51]. Level-3 kernels benefit the most because they perform a cubic number of operations on a quadratic amount of data, so each

entry loaded from memory is reused many times.

1.4.2 Sparse matrices

In optimization problems, the matrices $\mathbf{P}$ and $\mathbf{A}$ often contain mostly zero entries. Such matrices are called sparse, and conic solvers expect their problem data in sparse formats. Techniques that store and compute with only the nonzero entries date back to the linear programming computations of the 1950s and the work of Markowitz [52]. The simplest sparse matrix data structure is the coordinate or triplet format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of nonzero entries in the matrix. For example, the matrix

$$\mathbf{S} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix} \tag{1.10}$$

can be represented in coordinate format as

```
int    rows[] = {0, 1, 1,  1,  2,  2};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we use zero-based indexing and array declaration notation from the C programming language [53]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices. The compressed sparse row (CSR) format is commonly used instead.

The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (1.10) can be represented in CSR format as

```
int  idxptr[] = {0, 1,         4,      6};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

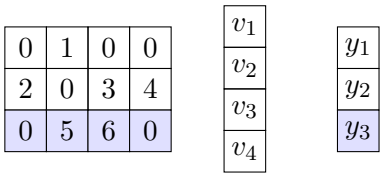
where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very cache-friendly because the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for iterative methods to solve sparse linear systems [54].

The compressed sparse column (CSC) format applies the same compression to the columns instead. The pointer array `idxptr` now points into a `rows` array, and the nonzero entries are stored column by column. The CSC arrays of a matrix read exactly as the CSR arrays of its transpose, so transposition is a conversion between the two formats, computable without sorting in one counting pass over the nonzeros [55]. Accessing a column of a CSR matrix instead requires a scan of every row, so algorithms that traverse columns prefer CSC; direct methods for sparse linear systems are the standard example [56].

The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [57] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

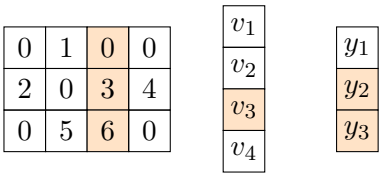
1.4.3 Sparse matrix products

Both compressed formats multiply a dense vector in one pass over the nonzeros, but the loop must follow the storage. CSR gathers, computing each output entry as a sparse dot product along its row, while CSC scatters, accumulating each column's contribution into the output vector. Figure 1.6 traces one product of the matrix in (1.10) under both formats.



$y_3 = 5v_2 + 6v_3$

(a) CSR gathers along a row



$y_2 \mathrel{+}= 3v_3, \quad y_3 \mathrel{+}= 6v_3$

(b) CSC scatters a column

Figure 1.6 The product $\mathbf{y} = \mathbf{S}\mathbf{v}$ with the matrix of (1.10). In CSR, the nonzeros of one row form a dot product that produces one output entry. In CSC, the nonzeros of one column are scaled by one input entry and accumulated into several output entries.

The product of two sparse matrices is formed row by row with Gustavson's algorithm [55]. Each row of the result is a combination of rows of the second operand, weighted by the nonzeros of the corresponding row of the first. Because the sparsity pattern of the result is not known in advance, the row is summed in a dense work array and then written back to sparse form [58]. It is the standard sparse matrix product, implemented in popular packages such as `Eigen` [59] and `scipy.sparse`.

In contrast to the BLAS kernels of §1.4.1, sparse products access memory irregularly. The

loops traverse the index arrays, so every nonzero costs an index load and an indirect access. Contiguous reads, SIMD, and cache tiling do not apply. Sparse formats are also a poor fit for dense data. A dense block stored in coordinate format triples its storage, one index pair per entry, and multiplying it through the sparse loop is far slower than the equivalent BLAS call. A sparse BLAS standard was specified by the BLAS Technical Forum [60], but implementations are few and far less widely adopted than the dense BLAS.

1.5 Contributions

We make the following contributions:

1. We build `SparseDiffEngine`, a C library with Python bindings that assembles sparse Jacobians and Hessians directly from the expression structure, with no sparsity detection or graph colouring.
2. We implement a canonicalization backend for `CVXPY` built on `SparseDiffEngine`; this backend avoids the large upfront cost of the parameter tensor while still caching computations for subsequent solves.
3. We benchmark `SparseDiffEngine` against `CVXPY`'s existing canonicalization backends.

All code contributions are open source and available at

`https://github.com/SparseDifferentiation/SparseDiffengine`
`https://github.com/SparseDifferentiation/Jacobian-accumulation`

The rest of this thesis is organized as follows. Chapter 2 reviews how `CVXPY` and other modelling languages compute the problem data. In Chapter 3 we present an overview of the main data structures and algorithms in `SparseDiffEngine` as well as its interface in `CVXPY`. Finally, Chapter 4 benchmarks this engine against `CVXPY`'s three canonicalization backends, and finally, we conclude with a summary of our contributions and future improvements in Chapter 5.

CHAPTER 2 LITERATURE REVIEW

This chapter reviews the implementation of `CVXPY`'s matrix stuffing (§2.1) and of other modelling language approaches, from sparse AD such as `CasADi` (§2.2) to coefficient extraction in `AMPL` and `JuMP` (§2.3). These modelling languages all produce the same output as `CVXPY`, the solver problem data, but differ in the inputs they accept and in how they compute the coefficients. For example, `CasADi` accepts vectorized expressions as input, but differentiates them through matrix–vector products, so the structure of the derivatives is never materialized and must be detected at runtime, over many sweeps of the expression graph. `AMPL` and `JuMP` let the user declare vectorized expressions but scalarize them when processing the data into coefficients. On the other hand, the canonicalization backends of `CVXPY` keep the expressions vectorized and need no detection step (for this reason, we only compare against these backends in Chapter 4).

2.1 Matrix stuffing in `CVXPY`

Matrix stuffing converts the conic-form expression trees into the problem data. After canonicalization every expression is affine, so the Jacobian of each node is a constant matrix, which matrix calculus [61] gives in closed form. Computing the problem data reduces to composing these constant Jacobians over the tree. This section reviews the derivative rules along with two design choices of the `CVXPY` implementation: the storage of the coefficients during the pass, and the caching of the problem data across parameter changes. Finally, we describe `CVXPY`'s current backends and their limitations.

Matrix calculus extends differentiation to functions whose arguments and values are vectors and matrices. Systematic rules for such derivatives date back to the 1940s [62]. The simplest convention is to vectorize [61]: every expression is identified with

$$\mathbf{vec}(\mathbf{X}) = \begin{bmatrix} x_{11} & \cdots & x_{m1} & x_{12} & \cdots & x_{mn} \end{bmatrix}^\top, \tag{2.1}$$

the vector that stacks the columns of $\mathbf{X} \in \mathbb{R}^{m \times n}$, and the derivative of one vectorized expression with respect to another is an ordinary Jacobian matrix. Under this convention, matrix operations become linear operators on the vectorized arguments, and their Jacobians are structured. For example, the matrix product $\mathbf{FX}$ has the Kronecker-structured, block-diagonal Jacobian $\mathbf{I} \otimes \mathbf{F}$ (Figure 2.1).

$$\texttt{X = cp.Variable((2, 2))} \quad \Longrightarrow \quad \frac{\partial\, \mathbf{vec}(\mathbf{X})}{\partial\, \mathbf{vec}(\mathbf{X})} = \begin{array}{c} \\ x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

(a) A matrix variable is the identity map of its own entries.

$$\texttt{Y = F @ X} \quad \Longrightarrow \quad \frac{\partial\, \mathbf{vec}(\mathbf{Y})}{\partial\, \mathbf{vec}(\mathbf{X})} = \mathbf{I} \otimes \mathbf{F} = \begin{array}{c} \\ y_{11} \\ y_{21} \\ y_{12} \\ y_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} f_{11} & f_{12} & 0 & 0 \\ f_{21} & f_{22} & 0 & 0 \\ 0 & 0 & f_{11} & f_{12} \\ 0 & 0 & f_{21} & f_{22} \end{bmatrix} \end{array}$$

(b) The Jacobian of the product $\mathbf{Y} = \mathbf{FX}$ is block-diagonal, one copy of $\mathbf{F}$ per column of $\mathbf{X}$.

Figure 2.1 Jacobians under the vectorization (2.1), for a matrix variable $\mathbf{X}$ and the product $\mathbf{FX}$, with $\mathbf{X}, \mathbf{F} \in \mathbb{R}^{2\times 2}$. The entries of $\mathbf{vec}(\mathbf{X})$, in column-major order, index the columns of the Jacobians.

2.1.1 Composing linear operators

Every node of a conic-form tree, not only the root, computes an affine function of the problem variables and can be written in the vectorized form

$$f(\mathbf{z}) = \mathbf{A}_f\, \mathbf{z} + \mathbf{b}_f = \begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} \begin{bmatrix} \mathbf{z} \\ 1 \end{bmatrix}, \tag{2.2}$$

where $\mathbf{z}$ is the stacked vector of all the problem variables, $\mathbf{A}_f$ is the Jacobian of f with respect to $\mathbf{z}$, and $\mathbf{b}_f$ is a constant vector. Stacking the augmented matrices of the constraint trees yields the problem data $[\mathbf{A}\ \mathbf{b}]$ of (1.2).

Matrix stuffing computes the augmented matrix in one bottom-up pass over the tree. A variable leaf contributes an identity block and a constant leaf contributes its value to the last column (Figure 2.3). An interior node with a single argument applies a constant linear operator $\mathbf{L}$ to the argument, which has the affine form (2.2) with augmented matrix $[\mathbf{A}_1\ \mathbf{b}_1]$; applying $\mathbf{L}$ gives

$$\begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} = \mathbf{L} \begin{bmatrix} \mathbf{A}_1 & \mathbf{b}_1 \end{bmatrix}. \tag{2.3}$$

The operator $\mathbf{L}$ is the Jacobian of the node with respect to its argument, so (2.3) is exactly the chain rule of matrix calculus. A node with k arguments applies one operator per argument,

$f = \sum_{i=1}^{k} \mathbf{L}_i f_i$, which gives

$$\begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} = \sum_{i=1}^{k} \mathbf{L}_i \begin{bmatrix} \mathbf{A}_i & \mathbf{b}_i \end{bmatrix}, \tag{2.4}$$

so the node costs one matrix product per argument.

In `CVXPY`, the linear operators divide into views and copies [63]. A view performs a simple manipulation of the coefficients of its argument. Atoms with one constant argument, such as `matmul` and `multiply`, make copies instead. The operator is a constant matrix built from the constant argument, and its product with the augmented matrix of the other argument is stored as a new coefficient matrix. Table 2.1 classifies a subset of the operators.

Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument f_1 with augmented matrix $[\mathbf{A}_1 \ \mathbf{b}_1]$, where $\otimes$ denotes the Kronecker product, $\odot$ the elementwise product, / the elementwise division, $\mathcal{I}$ an index set, and $\mathbf{diag}(\mathbf{v})$ the diagonal matrix with the entries of $\mathbf{v}$ on its diagonal.

Atom	**Operator L**	**Action on $[\mathbf{A}_1 \ \mathbf{b}_1]$**
Views		
$-f_1$	$-\mathbf{I}$	flip the sign of every stored entry
$f_1^\top$	$\mathbf{K}$, the vec-permutation	permute the rows
$f_1[\mathcal{I}]$	$\mathbf{P}_\mathcal{I}$, a row selection	keep the rows indexed by $\mathcal{I}$
reshape(f_1)	$\mathbf{I}$	none, the vectorization is unchanged
Copies		
$\mathbf{F} f_1$	$\mathbf{I} \otimes \mathbf{F}$	multiply by a block diagonal formed from $\mathbf{F}$
$f_1 \mathbf{B}$	$\mathbf{B}^\top \otimes \mathbf{I}$	multiply by a Kronecker matrix formed from $\mathbf{B}$
$\mathbf{C} \odot f_1$	$\mathbf{diag}(\mathbf{vec}(\mathbf{C}))$	multiply by a diagonal formed from $\mathbf{C}$
$f_1 / \mathbf{C}$	$\mathbf{diag}(\mathbf{vec}(\mathbf{C}))^{-1}$	multiply by the reciprocal diagonal

2.1.2 Example

We continue the example of §1.2.3, which canonicalized problem (1.5) into the conic-form trees of Figure 1.3. We now compute the problem data of these trees. The variables are first stacked into $\mathbf{z} = (t_1, t_2, \mathbf{x})$. Each variable then occupies a contiguous block of columns of $\mathbf{A}_f$, starting at its column offset within $\mathbf{z}$. The columns of the variables that do not appear in the expression f are zero, because f does not depend on their entries. Figure 2.2 gives an example of column offsets on another set of variables.

Matrix stuffing then runs bottom-up over the objective and the three constraint trees (Figure 2.3). Each leaf of $\mathbf{x}$ contributes an identity block of size n at the column offset of $\mathbf{x}$, and the constant leaf $\mathbf{g}$ contributes its value to the last column. The interior nodes apply their

$$\mathbf{z}^\top = \begin{array}{c} \begin{array}{ccccccc} \multicolumn{2}{c}{\mathbf{u}} & & \multicolumn{2}{c}{\mathbf{v}} & & \mathbf{w} \end{array} \\ \begin{array}{ccccccc} 0 & & & 2 & & & 5 \end{array} \\ \begin{bmatrix} u_1 & u_2 & v_1 & v_2 & v_3 & w_1 & w_2 \end{bmatrix} \end{array}$$

Figure 2.2 The column offsets of a stack of three variables $\mathbf{z} = (\mathbf{u}, \mathbf{v}, \mathbf{w})$ with $\mathbf{u} \in \mathbb{R}^2$, $\mathbf{v} \in \mathbb{R}^3$, and $\mathbf{w} \in \mathbb{R}^2$. Each offset is the total dimension of the preceding variables: $\mathbf{u}$ starts at column 0, $\mathbf{v}$ at 2, and $\mathbf{w}$ at 5. The entries of $\mathbf{z}$ index the columns of every augmented matrix $[\mathbf{A}_f \ \mathbf{b}_f]$.

operators to the matrices of their children; the product node $\mathbf{Fx}$, for example, multiplies the identity block of its argument by $\mathbf{F}$. Finally, the root stacks the augmented matrices of its children at their constraint row offsets into $[\mathbf{A}\ \mathbf{b}]$.

The augmented matrices of (2.2) can be stored in two ways (Figure 2.4). In the global layout, drawn in Figure 2.3, each node stores one matrix whose columns span the whole stacked vector $\mathbf{z}$, with the column offsets applied from the start. In the local layout, which `CVXPY` uses [63], each node keeps one coefficient block per variable in its subtree, and one for the constant data. The offsets are applied only after the pass, when each block is placed at its variable's columns and the blocks are concatenated into the full matrix $[\mathbf{A}\ \mathbf{b}]$. The local layout stores only the columns of the variables present in a subtree; the tradeoff is that each interior node applies its operator once per stored block, rather than once per argument.

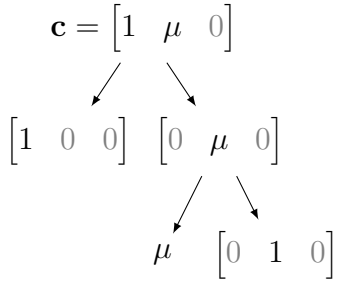
2.1.3 Parameter caching

In a problem family the structure is fixed and only the numeric parameter values change between solves. Under additional rules on how the parameters enter the expressions, known as Disciplined Parameterized Programming (DPP), the problem data is affine in the parameters [26]. To keep track of the parameter dependence, `CVXPY` adds a third dimension to the problem data, indexed by the parameter entries. Every coefficient matrix throughout the bottom-up pass becomes a tensor. At the root nodes we thus get the cached parameter maps

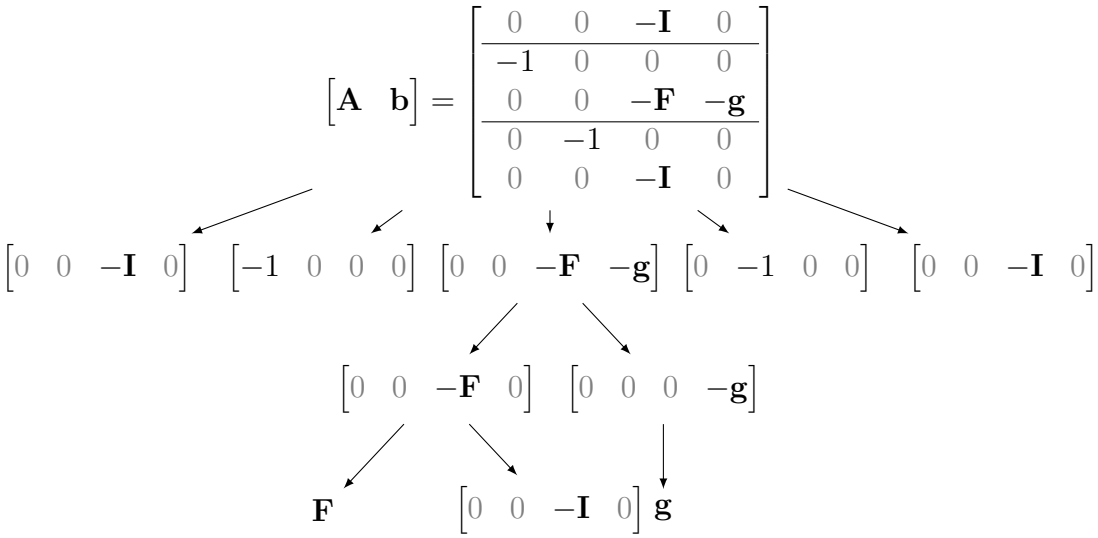
$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \mathbf{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}, \tag{2.5}$$

where $\mathbf{Q} \in \mathbb{R}^{n \times (p+1)}$ and $\mathbf{R} \in \mathbb{R}^{m(n+1) \times (p+1)}$. The objective map $\mathbf{Q}$ is shown in Figure 2.5 for problem (1.5). After a parameter change, a re-solve costs one sparse matrix–vector product per map.

Problems that do not conform to the DPP grammar, or that opt out of it with `ignore_dpp=True`, take a different approach to matrix stuffing. The reduction `EvalParams` replaces each pa-

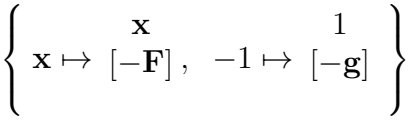


(a) Matrix stuffing of the objective

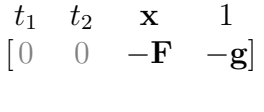


(b) Matrix stuffing tree of the constraints

Figure 2.3 Matrix stuffing of problem (1.5) by a bottom-up pass over the conic-form expression trees of Figure 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children. The coefficients appear negated because (1.2) writes the constraints as $\mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}$. The horizontal rules in $[\mathbf{A}\ \mathbf{b}]$ separate the rows of the three constraints.



(a) Local layout.



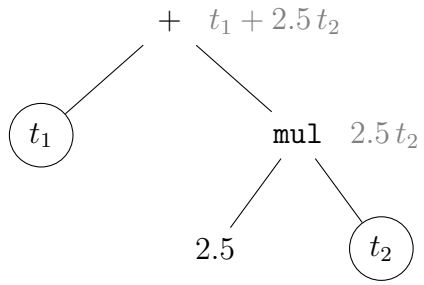
(b) Global layout.

Figure 2.4 What each layout stores during the pass, at the node $-(\mathbf{F}\mathbf{x} - \mathbf{g})$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns.

$$\mathbf{Q} = \begin{array}{c} \\ t_1 \\ t_2 \\ \mathbf{x} \end{array} \begin{array}{c} \mathbf{g}^\top \quad \mu \quad 1 \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}} = \begin{bmatrix} 1 \\ \mu \\ 0 \end{bmatrix}$$

Figure 2.5 The DPP path on the objective data of problem (1.5): the parameter map $\mathbf{Q}$ is built and cached once, and each re-solve recovers $\mathbf{c}$ with the single sparse product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$.

rameter leaf by a constant holding its current value, and the resulting parameter-free tree goes through the pass as if the problem had no parameters (Figure 2.6). The result is the same objective data $\mathbf{c}$ as in Figure 2.5, now obtained by repeating the whole matrix stuffing instead of applying the cached product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$. The `ignore_dpp` path is more general, as it supports arbitrary functions of the parameters.



$$\Rightarrow \quad \mathbf{c} = \begin{bmatrix} 1 & 2.5 & 0 \end{bmatrix}$$

Figure 2.6 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the pass is repeated on the parameter-free tree on every change of the parameters.

`CVXPY` implements the pass in three canonicalization backends that differ in how they represent the parameter axis [64]:

- the C++ backend `cvxcore` keeps one sparse matrix per parameter slice and loops over the slices at every operator;
- the `SciPy` backend stacks the slices vertically into a single strided two-dimensional sparse matrix;
- the `COO` backend does not form the tensor explicitly, and instead stores each nonzero with an explicit parameter index alongside its row and column.

When canonicalization remains too slow, `CVXPYgen` can compile the problem ahead of time

[65]. It emits problem-specific C code that embeds the cached parameter maps (2.5) of a DPP problem.

The pass of this section produces the affine data $\mathbf{c}$ and $[\mathbf{A}\ \mathbf{b}]$. For a quadratic objective, `CVXPY` replaces the affine argument of each quadratic atom by a new variable, adding the corresponding equality constraint; the objective becomes a sum of quadratic forms with constant matrices. A second matrix stuffing pass then assembles these matrices into the diagonal blocks of the matrix $\mathbf{P}$ of (1.2) [63].

All three backends share two limitations. Firstly, they delegate their matrix operations to a general-purpose sparse library, Eigen [59] in `cvxcore` and `scipy.sparse` [57] in the Python backends. The storage formats and kernels are fixed by the library, so structure known at canonicalization time cannot be exploited. In `cvxcore` the views of Table 2.1 also materialize explicit sparse operators. For example, a reshape builds a sparse identity matrix and multiplies the coefficients by it.

Secondly, the canonicalization backends store dense constant matrices as sparse matrices. When dense blocks appear in the problem data, they are both stored in an inconvenient format and computed without the optimized BLAS kernels of §1.4.1. Our approach in Chapter 3 addresses both limitations.

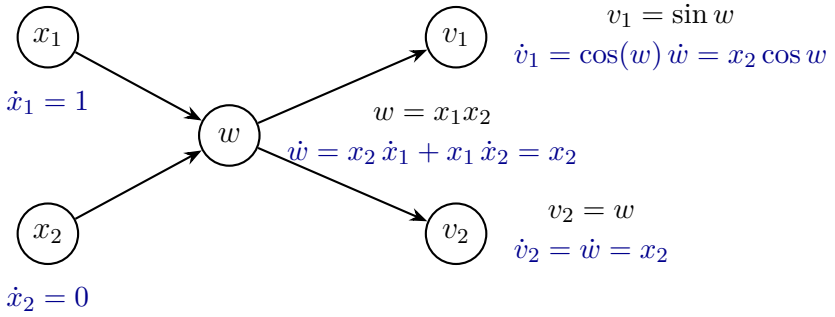
2.2 Sparse derivatives from detected structure

This section reviews methods that compute the full sparse Jacobian of a vector-valued function, given as an arbitrary program in a general-purpose language. No modelling language declares the structure of the program, so the sparsity of the derivatives must be detected [67]. Modern surveys of AD address dense derivatives, or the gradient of a scalar function, which reverse mode computes in a single sweep over the computational graph [40, 66]. In those settings, AD refers to the evaluation of matrix–vector products with the Jacobian or the Hessian, and the explicit matrix is never formed. Factorizations, direct linear solves, and many optimization solvers instead require the explicit matrix [67]. The methods of this section recover the full sparse matrix from multiple matrix–vector products. To reduce the number of products, graph colouring groups the vectors that can be propagated together, so that one product recovers several columns at once.

2.2.1 Matrix-vector products and accumulation

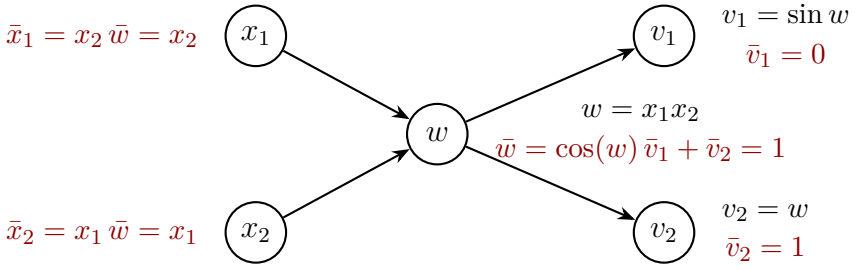
AD computes one matrix–vector product with the derivative per sweep of the computational graph, a traversal that applies the chain rule node by node at about the cost of one function

evaluation [46]. Forward mode starts from a seed vector $\mathbf{v}$ at the inputs and propagates tangents, and the Jacobian-Vector Product (JVP) $\mathbf{Jv}$ appears at the outputs. Reverse mode starts from a seed $\mathbf{w}$ at the outputs and propagates adjoints, and the Vector-Jacobian Product (VJP) $\mathbf{J}^\top\mathbf{w}$ appears at the inputs (Figures 2.7 and 2.8). For the Hessian, the analogous Hessian-vector product $\mathbf{Hv}$ comes from one forward sweep over one reverse sweep.



$$\begin{bmatrix}\dot{v}_1\\ \dot{v}_2\end{bmatrix} = \begin{bmatrix}x_2\cos w\\ x_2\end{bmatrix} = \mathbf{J}\mathbf{e}_1, \qquad \mathbf{J} = \begin{bmatrix}x_2\cos w & x_1\cos w\\ x_2 & x_1\end{bmatrix}.$$

Figure 2.7 Forward sweep of the computational graph of $F(x_1, x_2) = (\sin(x_1x_2),\ x_1x_2)$, with the intermediate value $w = x_1x_2$. Alongside its value, each node carries one tangent $\dot{v}$ (in blue), updated by the chain rule following the order of evaluation. The seed $(\dot{x}_1, \dot{x}_2) = (1, 0)$ delivers the first column of $\mathbf{J}$ at the outputs.

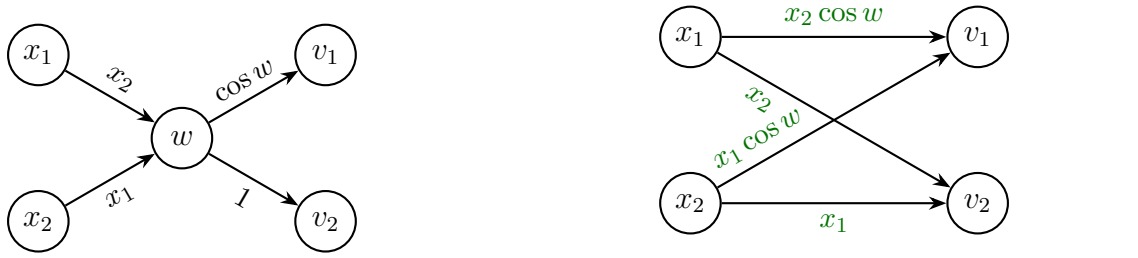


$$\begin{bmatrix}\bar{x}_1 & \bar{x}_2\end{bmatrix} = \begin{bmatrix}x_2 & x_1\end{bmatrix} = \mathbf{e}_2^\top\mathbf{J}.$$

Figure 2.8 Reverse sweep of the computational graph of Figure 2.7. Each node carries one adjoint $\bar{v}$ (in red), updated by the chain rule against the order of evaluation. The seed $(\bar{v}_1, \bar{v}_2) = (0, 1)$ delivers the second row of $\mathbf{J}$ at the inputs.

Recovering the full matrix takes a sequence of sweeps, one per seed vector. The n unit seed vectors recover $\mathbf{J}$ column by column in forward mode, and the m unit seed vectors recover it row by row in reverse mode [46]. The other approach is accumulation, which combines the

local derivatives of the graph directly into **J** by eliminating one intermediate vertex at a time (Figure 2.9) [68]; finding the optimal elimination order is NP-complete [46, 69]. Batching the n unit seeds into one forward sweep gives each node a whole tangent, the derivative of that node with respect to all the inputs. Eliminating the vertices in the order of evaluation stores the same derivative at each node, so the batched sweep and forward-order elimination perform the same computation.



(a) Each edge carries a local derivative.

(b) After eliminating w, the edges are the entries of **J**.

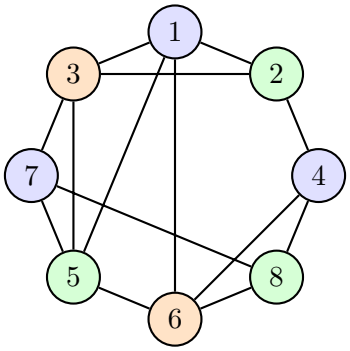
Figure 2.9 Accumulation of the Jacobian by vertex elimination, on the graph of Figure 2.7. Eliminating the intermediate vertex w connects each of its predecessors to each of its successors and multiplies the local derivatives along the path (in green); parallel edges, when they arise, are summed. Once every intermediate vertex is eliminated, the graph is bipartite and its edges are the entries of **J**.

In practice, AD tools such as ADOL-C [70] and CppAD [71] record the scalar graph as a runtime trace, one record per elementary operation [46, 72]. A single matrix–vector product unrolls into one scalar multiplication for every entry of the matrix, together with the additions that sum them, so the trace of a realistic problem can sometimes reach millions of nodes. On such a trace, every intermediate vertex would store its derivative with respect to all the inputs, so accumulation is rarely used in practice. When **J** is sparse, most of each sweep is then wasted on zeros, and the cost of recovering the matrix scales with its dimension rather than with its number of nonzeros.

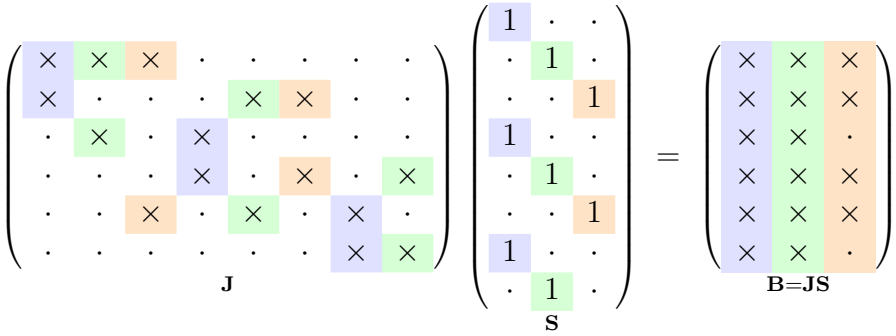
2.2.2 Compression by graph colouring

Several columns of a sparse Jacobian can share one seed [28]. Two columns with no nonzero in a common row are structurally orthogonal, and one sweep recovers both, each entry landing in a distinct row of the product. Finding the fewest such groups was later formalized as a colouring of the column intersection graph, which has one vertex per column and an edge between two columns that share a row [29, 73]. Figure 2.10 colours the graph of a 6×8 Jacobian and recovers the matrix in three sweeps instead of eight. The approach extends

to the Hessian, which compresses further because its symmetry allows dedicated star and acyclic colourings [74, 75].



(a) The column intersection graph of **J**, with one vertex per column and an edge between two columns that share a row, coloured with three colours.



(b) The seed matrix **S** sums the unit vectors of each colour class, and the compressed product **B** = **JS** recovers every nonzero of **J**, each in a distinct row of its compressed column.

Figure 2.10 Graph colouring compresses the seeds. Colouring the column intersection graph groups the columns of the sparse Jacobian **J** into three structurally orthogonal classes, and the matrix is recovered in three sweeps instead of eight.

The sparse Jacobian is obtained from the products in three stages: detect the sparsity pattern, compress it by colouring, and propagate the compressed seeds of Figure 2.10 through the graph to recover the nonzero entries [67]. Detection propagates sparsity patterns instead of values through the same graph, replacing each arithmetic operation by a union of index sets [46]. The pattern and its colouring are computed once and reused, so their cost is amortized across repeated evaluations of the same function.

2.2.3 CasADi

CasADi [12] is an open-source framework for nonlinear optimization and optimal control built around AD. It is widely used in academia and industry, notably for model predictive control, trajectory optimization, robotics, and energy systems [77]. Its core is written in C++ and is used from Python, MATLAB, or Octave. It provides the building blocks of gradient-based optimization: symbolic expressions, derivatives, integrators, and interfaces to NLP and QP solvers. The gradients, Jacobians, and Hessians these solvers require are obtained by differentiating the model.

CasADi supports two types of expressions, and both return the matrix derivatives in a sparse format, which is expected by most solvers [77]. An SX graph is scalar, with exact node-by-node sparsity, but its size grows with the problem data. An MX graph is vectorized. Its nodes are whole matrix operations, so the graph stays small even when the model is large. Either graph can be exported as self-contained C code and compiled ahead of time for embedded applications.

CasADi computes its derivatives by the detection, colouring, and batched sweeps of this section, and caches the pattern, the colouring, and the compiled derivative function across evaluations. The following listing rebuilds the conic-form trees of Figure 1.3, obtained after canonicalization, as MX expressions; differentiating them recovers the problem data that matrix stuffing assembles in §2.1.2.

```
import casadi as cs
t1, t2 = cs.MX.sym("t1"), cs.MX.sym("t2")
x = cs.MX.sym("x", n)
g, mu = cs.MX.sym("g", m), cs.MX.sym("mu")
z = cs.vertcat(t1, t2, x)               # stacked variables
expr = cs.vertcat(x, t1, F @ x - g, t2, x) # stacked affine expressions
A = -cs.jacobian(expr, z)               # the matrix A
b = cs.substitute(expr, z, 0)           # the vector b
c = cs.gradient(t1 + mu * t2, z)        # the objective vector c
```

The Jacobian with respect to the stacked variable $\mathbf{z}$ recovers the matrix $\mathbf{A}$ of Figure 2.3 entry by entry; differentiation is the equivalent of matrix stuffing, with the coefficients obtained by those sweeps rather than by a bottom-up pass over the tree. The parameters remain symbols, so $\mathbf{b}$ and $\mathbf{c}$ are expressions of $\mathbf{g}$ and μ, re-evaluated for each parameter value. This is how the qpsol interface builds its problem data internally, for problems with a quadratic objective and affine constraints. A model written at the same high level as in CVXPY, with the norms of (1.5) kept in the expression graph, is solved through nlpsol instead. The same

sweeps assemble the gradient, constraint Jacobian, and Hessian of the Lagrangian that the NLP solver queries at each iteration.

2.3 Coefficient extraction from declared structure

The systems of the previous section detect the structure of the problem at runtime and recover its coefficients through colouring. `AMPL` and `JuMP` need no detection, because the structure is declared up front and the coefficients are stored as the user types them; extraction reduces to reading the stored coefficients back (Table 2.2).

2.3.1 `AMPL`

`AMPL` [5] is the oldest system in this review, and its architecture still defines how most solvers receive nonlinear problems. The AMPL translator runs in its own process. It parses the model and the data, instantiates every constraint, and writes the resulting instance to a `.nl` file [30], which the solver, a separate executable linking the AMPL Solver Library (ASL), reads back [31].

The `.nl` file is an instance format rather than a model format. Sets and indexing are resolved by the translator, and what remains is one flat record per objective and constraint. A header states the dimensions of the problem and the nonzero counts of its Jacobian and gradients; the body is a sequence of segments keyed by a leading letter. `C` and `O` segments hold the nonlinear parts of constraints and objectives as expression graphs in Polish prefix notation; `J` and `G` segments hold their linear parts as explicit lists of (variable, coefficient) pairs; the remaining segments hold bounds and nonzero counts.

The following quadratic program shows the syntax of `.nl` files.

```
var x; var y;
minimize obj: x^2 + x*y + 3*x;
subject to c1: x + 2*y >= 1;
```

The translator writes the following `.nl` file (ASCII encoding; annotations and elisions ours):

```
g3 0 1 0    # text format; 9 more header lines follow
 2 1 1 0 0  # 2 vars, 1 constraint, 1 objective, 0 ranges, 0 eqns
...
r           # constraint bounds: c1 >= 1
2 1
...
O0 0        # minimize objective 0; nonlinear part x^2 + x*y
```

```
o0          #   add
o5          #     pow
v0          #       variable 0 (x)
n2          #       constant 2
...         #     multiply subtree of x*y
J0 2        # linear Jacobian row of c1: two entries
0 1         #   coefficient 1 on variable 0
1 2         #   coefficient 2 on variable 1
G0 2        # linear part of the objective gradient: 3x
0 3
1 0
```

The quadratic terms $x^2 + xy$ appear only as the expression graph of the `O0` segment; every linear coefficient is spelled out in the `J` and `G` segments. For a linear program, the file therefore contains the complete problem data.

Inside the solver, ASL reads the linear coefficients and bounds directly from the file, so for a linear or quadratic-over-linear instance extraction consists only of this read, at a cost linear in the number of nonzeros. Only the rebuilt nonlinear graphs are differentiated, by reverse-mode AD [78], with sparse Hessians assembled from partial separability [79].

The format has three benefits: any system that writes `.nl` reaches every solver that links ASL; the format has stayed backward compatible since 1990; and an instance is an ordinary file that can be archived, shipped to reproduce a bug, and rerun across machines. The decoupling also has a cost. Every solve serializes the instance to disk and rebuilds its expression graphs in the solver process.

2.3.2 `JuMP`

`JuMP` [11, 80] is an algebraic modelling language embedded in Julia; its abstraction layer between models and solvers is MOI [32]. Where `AMPL` decouples modeller and solver through a file, `JuMP` decouples them through a typed in-memory interface.

The macros of `JuMP` rewrite each expression as it is entered. `@objective` and `@constraint` build affine and quadratic expression objects term by term, merging duplicates eagerly, so the model stores coefficient lists instead of an expression tree. These lists are typed by MOI. A `VectorAffineFunction` is a list of (row, variable, coefficient) terms plus a constant vector; a `ScalarQuadraticFunction` adds a list of (variable, variable, coefficient) triples. A constraint pairs one such function with a set, for instance a nonnegative orthant or a second-order cone. This is the functional conic form of §1.2.1. The constraint of the `AMPL` example, entered as

`@constraint(model, x + 2y >= 1)`, is stored as the pair

```
ScalarAffineFunction([(1.0, x), (2.0, y)], 0.0) in GreaterThan(1.0)
```

a term list, a constant, and a set; the expression itself is not retained after the macro returns.

When the model is passed to a solver, MOI copies each constraint into the solver's own representation (`copy_to`). Solvers that consume matrix data use the `MatrixOfConstraints` utility, which concatenates the stored term lists into one sparse matrix in the solver's format. The cost of extraction is the cost of this copy, which is linear in the number of terms and requires no analysis of any graph. The assembled matrix is the same problem data that `CVXPY`'s matrix stuffing computes (§2.1); `JuMP` arrives at it by copying stored coefficients rather than by differentiating conic-form trees.

When a solver does not support a constraint type natively, MOI reformulates it through its *bridge* system. A bridge is a structural rewrite from one function-set pair to another, for instance from a rotated to a plain second-order cone, chosen by a shortest-path search over the graph of available bridges [32]. Bridging manipulates stored coefficients and adds auxiliary variables and constraints. It is similar to `CVXPY`'s canonicalization (§1.2.2), except that `JuMP` rewrites problems whose data is already in matrix form, while `CVXPY` must first produce that data from a DCP atom tree; producing this data is the work this thesis accelerates.

`JuMP` also has a nonlinear path, in which expression graphs are retained and derivatives come from sparse AD in the style of §2.2; it serves general nonlinear solvers and is out of scope for this thesis.

Table 2.2 The reviewed systems and the engine of Chapter 3, sorted by the origin of their problem-data coefficients: stored as the user typed them, detected at runtime, or differentiated out of an expression graph.

System	Coefficients	Sparsity known from	Extraction cost
`JuMP`	stored	the term lists themselves	one copy of the term lists (`copy_to`)
`AMPL`	stored[a]	partial separability of the scalar tape	`.nl` write and ASL read
Hill and Dalle [67]	detected	index-set tracing, then colouring	compressed sweeps
`CasADi` [12]	detected	pattern propagation, then colouring	compressed sweeps
`CVXPY` [19]	differentiated	emerges from generic sparse products	operator pass over the conic-form trees
This thesis	**differentiated**	**analytic, per DCP atom**	**operator pass over the conic-form trees**

[a] Linear parts only; the nonlinear parts are differentiated by ASL in the solver process.

CHAPTER 3 A LIGHTWEIGHT CANONICALIZATION ENGINE

Mathematically, `SparseDiffEngine` computes the same derivatives as `CVXPY`'s canonicalization backend: both apply the chain rule of matrix calculus to trees of vectorized expressions, producing the Jacobian and Hessian of §1.3. The differences are in the implementation details, which we will present in this chapter. Firstly, the use of a custom sparse matrix data structure allows a natural split between the symbolic and numeric phases of the derivative computation. This separation is particularly useful for compiling problems involving parameters, where the structure is fixed but the numerical values can change. Secondly, in addition to being sparse, the data of most optimization problems also have a lot of structure, such as dense blocks or diagonality. In `SparseDiffEngine` we introduce two new matrix classes: `permuted-dense` and `stacked-permuted-dense` for this purpose; they enable us to make use of dense storage and BLAS subroutines when possible.

Finally, some important design decisions differ from `CVXPY`'s backends for simplicity rather than speed. Every node carries a single Jacobian whose columns span all of the variables of the problem, rather than a dictionary of local Jacobians keyed by variables. As a consequence, the chain rule is a single matrix product, and the constraint matrix is built by stacking the Jacobians of the constraints. In addition, quadratic functions pass through dedicated Hessian oracles instead of a separate substitution and matrix stuffing pass.

The rest of the chapter is structured as follows: we begin with a brief overview of the engine, describing how expressions are represented and how the code is organized. We then present the matrix representations used for the derivative computations, where we will focus much of our attention on the `permuted-dense` matrix, which appears most often. Next we describe the algorithms, or kernels, which allow us to operate on and combine these matrix representations. Rather than attempting to explain too much, we focus on the organization of the code and on how we handle operations between different types of matrices. An intrigued reader will learn more from exploring the repository, which is open source. The chapter ends with some comments about the Application Programming Interface (API) of `SparseDiffEngine` as well as how it is used within `CVXPY`.

3.1 Overview of the engine

`SparseDiffEngine` is a small C library which is around seventeen thousand lines of code (and a similar number of lines for the test suite). The expression node is defined in `expr.h`;

atoms that carry extra data, such as the exponent of a power, define a larger structure in `subexpr.h`. Each atom is implemented in its own file in the `atoms/` directory; there are around forty-five of them, divided into six subdirectories based on their mathematical properties: `affine`, elementwise and bivariate functions with full or restricted domains, and a small group of `other` atoms such as the quadratic form. The matrix data structures and the computational kernels that operate on them are in `utils/`, and `problem.c` collects the objective and constraints of a problem and implements the oracles that are passed to a solver.

3.1.1 Atom classes

Atoms in the same subdirectory compute their derivatives in the same way (Table 3.1). In this chapter we only discuss the affine atoms and the quadratic form, because these are the only atoms that appear in canonicalized DCP problems; the remaining classes are used by the nonlinear programming (NLP) interface of `CVXPY` [34]. Each atom implements

Table 3.1 The atom classes and how each computes its Jacobian.

Class	Examples	Jacobian computation
affine	`sum`, `index`, `left_matmul`	fixed; index maps and copies
elementwise, full domain	`exp`, `power`, `logistic`	diagonal scaling of the child Jacobian
elementwise, restricted domain	`log`, `entr`, `atanh`	diagonal at the variable's columns
bivariate, full domain	`multiply`, `matmul`	two-term chain rule
bivariate, restricted domain	`quad_over_lin`, `rel_entr`	two-term chain rule
other	`quad_form`, `prod`	specific to each atom

three operations: a *forward* pass, which evaluates the atom at a point; a *symbolic* pass, which computes the sparsity patterns of its derivatives and allocates memory for them; and a *numeric* pass, which computes the numerical values inside the fixed patterns. The constructor of the atom stores pointers to these three functions in the expression node.

3.1.2 The expression node

An expression node stores its dimensions, pointers to at most two children, a buffer for its value, and its derivative matrices, whose `n_vars` columns span all of the variables of the problem. A simplified version of the definition in `expr.h` is shown below.

```
typedef struct expr
```

```
{
    int d1, d2, size, n_vars, refcount, var_id;
    struct expr *left;
    struct expr *right;

    double *value;   /* forward value             */
    matrix *jacobian; /* Jacobian w.r.t. the variables */

    forward_fn      forward;
    jacobian_init_fn jacobian_init_impl; /* symbolic pass */
    eval_jacobian_fn eval_jacobian;    /* numeric pass */
    /* ... Hessian oracle, workspace and caching fields ... */
} expr;
```

Trees are built bottom up, with one constructor function per atom; the Python bindings expose the same constructors, so a `CVXPY` expression is converted by a single traversal that calls the constructor of the corresponding atom at each node (§3.4). Expressions that appear multiple times can be shared, in which case the tree becomes a directed acyclic graph. Nodes are therefore reference counted, and the symbolic pass returns early if a node was already initialized through another parent.

There are two leaf nodes, variables and parameters. A *variable* occupies a contiguous slice of the global variable vector, starting at its offset `var_id`; its Jacobian is a selection matrix with one unit entry per row, written once during the symbolic pass. Unlike `CVXPY`, there is no separate constant node: constants and parameters share a single leaf type, distinguished by an identifier that is -1 for constants and, for updatable parameters, the offset of the parameter in the global parameter vector. The Jacobian of this leaf is empty, because parameters are fixed during a solve, and because parameters are leaves of the tree, the derivative structure remains valid when their values change (§1.1).

3.2 Matrix representations

The `jacobian` field of the expression node has a polymorphic type called `matrix`, with one representation for each kind of structure that we want to exploit. Each atom chooses the representation of its Jacobian when it allocates it during the symbolic pass; multiplication by dense data allocates a permuted dense matrix, for example. After that, every operation on the matrix, such as transposition, row indexing, or a chain-rule product, dispatches to a kernel specific to the representations involved. The default representation is a generic sparse

matrix stored in the CSR format of §1.4.2.

3.2.1 The permuted dense matrix

The Jacobians that appear during canonicalization are usually not sparse in the sense of having scattered, isolated nonzeros. Multiplication by problem data places the data itself, or a product involving it, as a dense block at a subset of the rows and columns of the Jacobian. Consider the expression $\mathbf{Bx}$ with dense data $\mathbf{B} \in \mathbb{R}^{50\times 50}$, in a problem with ten thousand variables. Its Jacobian with respect to the full variable vector is a matrix with ten thousand columns that contains $\mathbf{B}$ in the fifty columns of $\mathbf{x}$ and is zero everywhere else. If this Jacobian is stored in a generic sparse format, every subsequent product goes through sparse kernels, and the hardware efficiency of §1.4.1 is lost.

We therefore store such matrices in *factored form*: we keep the dense block and the index vectors that place it, and never assemble the global matrix. A *permuted dense* matrix is a triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$, where $\mathbf{X} \in \mathbb{R}^{m_0\times n_0}$ is a dense block and $\mathbf{r}$ and $\mathbf{c}$ are strictly increasing index vectors, representing

$$\mathbf{A} = \mathbf{S}_r^\top \mathbf{X} \mathbf{S}_c \in \mathbb{R}^{m\times n}, \qquad A_{r_i, c_j} = X_{ij}, \tag{3.1}$$

where $\mathbf{S}_r \in \{0,1\}^{m_0\times m}$ and $\mathbf{S}_c \in \{0,1\}^{n_0\times n}$ are the selector matrices whose rows are the rows of the identity indexed by $\mathbf{r}$ and $\mathbf{c}$: multiplying a vector by $\mathbf{S}_r$ selects the entries indexed by $\mathbf{r}$, and multiplying by $\mathbf{S}_r^\top$ scatters them back into a vector of length m, with zeros elsewhere. All other entries of $\mathbf{A}$ are structurally zero and never stored. Figure 3.1 shows an example with a 3×3 block inside a 6×7 matrix.

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{00} & \cdot & x_{01} & \cdot & \cdot & x_{02} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}$$

$$\mathbf{r} = (1, 3, 4), \qquad \mathbf{c} = (0, 2, 5).$$

Figure 3.1 A permuted dense matrix. The dense block $\mathbf{X}$ is stored once, contiguously; the index vectors $\mathbf{r}$ and $\mathbf{c}$ place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored. Each block row keeps its own shade, reused in Figure 3.2.

The block $\mathbf{X}$ is stored contiguously in row-major order and the index vectors as two integer

arrays, so storage does not depend on the global dimensions. Two inverse maps, which send a global index to its block index or to -1, make membership checks constant time. Structure is computed with integer arithmetic on the index vectors; floating point operations run only on the contiguous block.

Every operation we need maps a permuted dense matrix to another permuted dense matrix. Transposition swaps the two index vectors and transposes the block. The structural atoms, such as indexing, broadcasting, and the row partitions of `sum`, work the same way: because row r_i of $\mathbf{A}$ is row i of $\mathbf{X}$ and every other row is zero, an operation on the rows of $\mathbf{A}$ reduces to the same operation on the rows of $\mathbf{X}$, guided by the inverse row map.

output row i	0	1	2
requested global row s_i	3	0	4
inverse row map $\mathbf{r}^{-1}[s_i]$	1	-1	2
action	copy $\mathbf{X}_{1,:}$	zero row	copy $\mathbf{X}_{2,:}$

$$\mathbf{C} = \mathbf{A}[(3,0,4),\,:] \;=\; \begin{pmatrix} x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \end{pmatrix} \begin{matrix} \leftarrow \mathbf{A}_{3,:} \\ \leftarrow \mathbf{A}_{0,:} = 0 \\ \leftarrow \mathbf{A}_{4,:} \end{matrix}$$

$$\mathbf{X}_C \;=\; \mathbf{X}[(1,2),\,:] \;=\; \begin{pmatrix} x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}, \qquad \mathbf{r}_C = (0,2), \qquad \mathbf{c}_C = \mathbf{c} = (0,2,5).$$

Figure 3.2 Row indexing of the permuted dense matrix $\mathbf{A}$ of Figure 3.1. The table traces the scan of the inverse row map: each requested global row yields a block row of $\mathbf{X}$ to copy, or -1, in which case the output row is structurally zero and not stored. Shades follow the block rows of Figure 3.1. The result is the permuted dense triple $(\mathbf{X}_C, \mathbf{r}_C, \mathbf{c}_C)$.

Figure 3.2 traces the indexing of rows $(3,0,4)$ of the matrix in Figure 3.1: each hit in the inverse row map yields a block row to copy, and the miss at row 0 produces a zero row that is not stored. The result is again permuted dense, with block $\mathbf{X}[(1,2),\,:]$, row vector $(0,2)$, and the column vector of $\mathbf{A}$; a repeated row would copy the same block row twice. The structure costs one integer scan, the values two row copies.

The product of two permuted dense matrices is also permuted dense: for permuted dense $\mathbf{A}$ and $\mathbf{B}$, the chain-rule product $\mathbf{B}^\top\mathbf{A}$ factors as

$$\mathbf{B}^\top\mathbf{A} = \mathbf{S}_{c_B}^\top \mathbf{X}_B^\top \underbrace{\left(\mathbf{S}_{r_B}\mathbf{S}_{r_A}^\top\right)}_{\text{row-set intersection}} \mathbf{X}_A \mathbf{S}_{c_A}, \tag{3.2}$$

The middle factor is a zero-one matrix whose entry (k,l) is one exactly when $r_{B,k} = r_{A,l}$, so

the sparsity analysis of the product amounts to intersecting two sorted integer arrays. The result has row vector $\mathbf{c}_B$ and column vector $\mathbf{c}_A$, which are known before any numerical values exist. For the values there are three cases. If the two row sets are equal, the product is computed on the two blocks with a single call to `dgemm`, the double precision general matrix–matrix product of level-3 BLAS (§1.4.1). If they overlap partially, the matching rows are first gathered into a workspace (allocated during the symbolic pass), followed by the same `dgemm`. If they are disjoint, the product is zero, and nothing is computed in the numeric pass. The weighted product $\mathbf{B}^\top\mathbf{DA}$, needed for the Hessian of the objective, is handled the same way.

The main benefit of the factored form (3.1) is that the floating point work is done by dense kernels. If the same matrix is stored in the CSR format, its products go through sparse kernels, whose indexed accesses and irregular inner loops reach only a fraction of the throughput of BLAS [51] at the same operation count. In the factored form, the numeric pass of each kernel consists of a few BLAS calls on contiguous buffers. The product $\mathbf{A}^\top\mathbf{DA}$, for example, is computed with one copy, m_0 scalings, and one `dgemm`, and the cost does not depend on the global dimensions.

A permuted dense matrix cannot represent nonzeros outside its block: by (3.1), every such entry is structurally zero. A Jacobian that mixes a dense block with scattered entries uses the default CSR representation, and an operation whose result does not keep the block structure, such as adding operands with different patterns, is performed on CSR views of its operands. The remaining risk is dense data that contains many zeros, which would be stored and multiplied densely, spending floating point work on zeros; because `CVXPY` supplies sparse data in sparse formats, this does not arise in practice.

3.2.2 The stacked permuted dense matrix

When an expression with matrix shape $n \times p$ is multiplied by data $\mathbf{A} \in \mathbb{R}^{m\times n}$, the effect on the vectorized expression is that of the block diagonal operator $\mathbf{I}_p \otimes \mathbf{A}$; this is the `left_matmul` atom. The operator is never formed: the engine keeps one copy of $\mathbf{A}$ and applies it block by block. In memory, the p segments of length n of a vector $\mathbf{x}$ form a $p \times n$ row-major matrix $\tilde{\mathbf{X}}$, and by a standard vectorization identity [61],

$$\mathbf{y} = (\mathbf{I}_p \otimes \mathbf{A})\,\mathbf{x} \quad \Longleftrightarrow \quad \tilde{\mathbf{Y}} = \tilde{\mathbf{X}}\mathbf{A}^\top, \tag{3.3}$$

so applying the operator to a vector is a single `dgemm` on the stored $\mathbf{A}$. Avoiding the explicit operator made the initialization of the atom's sparsity pattern roughly one hundred times faster.

Identity (3.3) applies the operator; the atom must also store its Jacobian. The parent node receives $(\mathbf{I}_p \otimes \mathbf{A})\,\mathbf{J}$, where $\mathbf{J}$ is the Jacobian of the child, and transposes, indexes, and multiplies it like any other Jacobian. This matrix is made of p dense blocks at disjoint rows, one per column of the operand, and each block generally sits at its own global columns, so no single triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$ describes it. Merging the blocks into one larger triple would store the full block diagonal, p copies of $\mathbf{A}$ separated by blocks of zeros, and falling back to generic sparse would lose the dense kernels. The engine instead keeps the blocks as they are, in a third representation.

A *stacked permuted dense* matrix is a vertical union of permuted dense blocks whose row vectors are pairwise disjoint; the column vectors may overlap. The blocks share one value buffer. Every kernel treats the blocks one at a time: structural atoms rewrite the index vectors of each block, and products distribute over the blocks, with one `dgemm` for each pair of blocks with intersecting row sets and nothing at all for the pairs with disjoint row sets, as in (3.2). The assembled matrix is never allocated or traversed, and each stacked kernel is a short loop over the corresponding permuted dense kernel. Figure 3.3 shows the Jacobian of $\mathbf{AX}$ for a variable $\mathbf{X}$ with two columns: one block per column of $\mathbf{X}$, each a copy of $\mathbf{A}$ at its own rows and columns.

$$\mathbf{M} = \begin{pmatrix} a_{00} & a_{01} & \cdot & \cdot & \cdot & \cdot \\ a_{10} & a_{11} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & a_{00} & a_{01} & \cdot & \cdot \\ \cdot & \cdot & a_{10} & a_{11} & \cdot & \cdot \end{pmatrix}$$

$$\mathbf{X}_1 = \mathbf{X}_2 = \mathbf{A}, \qquad \mathbf{r}_1 = (0,1), \quad \mathbf{c}_1 = (0,1), \qquad \mathbf{r}_2 = (2,3), \quad \mathbf{c}_2 = (2,3).$$

Figure 3.3 The Jacobian of $\mathbf{AX}$ as a stacked permuted dense matrix, for dense $\mathbf{A} \in \mathbb{R}^{2\times 2}$ and a variable $\mathbf{X} \in \mathbb{R}^{2\times 2}$ in a problem with six variables. There is one block per column of $\mathbf{X}$, each a permuted dense copy of $\mathbf{A}$ at its own rows and columns. The 4×6 matrix $\mathbf{M}$ is never assembled; a product with $\mathbf{M}$ is one `dgemm` per block.

Generic sparse, permuted dense, and stacked permuted dense are the only three matrix representations in the engine. Generic sparse is the default: it is produced by the variable leaves and by every atom that does not hold dense data. Permuted dense and stacked permuted dense are created by the two atoms that hold dense data, `left_matmul` and the quadratic form, and propagate from there through the structural atoms and the chain rule.

3.3 Computational kernels

A *kernel* is an operation on one or two of the matrix representations of §3.2: a transposition, a row selection, a chain-rule product. Every kernel is split into an alloc function and a fill function, called by the symbolic and numeric passes respectively: alloc computes the structure of the result and reserves its memory, and fill writes numerical values into the fixed structure. This section describes the kernels from the point of view of the atoms that call them; §3.3.1 describes how a product selects a kernel when each operand can have any of the three representations, and §3.3.2 collects the work that is done once during the symbolic pass instead of at every evaluation.

3.3.1 Kernel dispatch

Unary operations are dispatched through the function pointer table of §3.2, based on the representation of their single operand. The chain-rule products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{D}\mathbf{A}$ depend on the representations of both operands, which a single table cannot express. They are instead resolved with double dispatch: a dispatch function switches on the type tags of $\mathbf{B}$ and then of $\mathbf{A}$, and calls one of nine kernels. Figure 3.4 shows the representation of the result for the nine combinations of operand types.

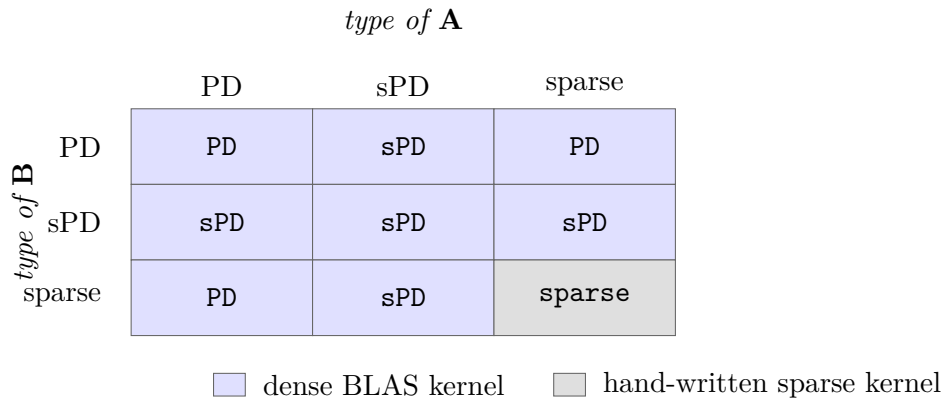


Figure 3.4 Double dispatch of the chain-rule products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{D}\mathbf{A}$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its colour indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel.

A product with a permuted dense or stacked permuted dense operand produces a result of the same kind, so once a dense block has been identified, it remains in a structured representation through all later products. Eight of the nine kernels are based on dense

BLAS calls; in the mixed combinations, the columns of the sparse operand are gathered into the dense computation. Only the product of two generic sparse matrices uses Gustavson's algorithm [55], with the symbolic and numeric phases discussed in §1.4.3.

3.3.2 Reusing work across passes

The Jacobian of an affine atom does not depend on $\mathbf{x}$: for the structural atoms, such as indexing, transposition, reshaping, stacking, and summation, it is determined by shapes and index sets alone, and the remaining affine atoms, such as `left_matmul`, `scalar_mult`, `convolve`, and `kron`, place problem data in the Jacobian, whose values only change when a parameter update sets their refresh flag. The quadratic form is the only atom outside the affine class that appears in canonicalized problems; it contributes the objective matrix $\mathbf{P}$ through the Hessian oracle (§3.4).

The numeric pass never allocates memory; every buffer, including the scratch space of the kernels, is allocated during the symbolic pass. Beyond allocation, several other kinds of work are done only once. Sparsity patterns are computed symbolically: `add` merges the patterns of its children with a sorted merge, the sparse products run the symbolic phase of Gustavson's algorithm [58], and the factored products of §3.2 determine their output index vectors before any values exist. Index maps are also precomputed: transposition stores its row permutation, `kron` its per-row copy maps, and `sum` the map from child nonzeros to output entries, so the numeric pass performs no index arithmetic. When the index vectors of two operands do not intersect, the product is recorded as empty during the symbolic pass, and its numeric pass does nothing. Finally, values that cannot change are written once; the Jacobian of a variable, for example, is filled during the symbolic pass and never touched again.

Figure 3.5 illustrates this for the `sum` atom. During the symbolic pass, the representation of the child allocates the output and fills `idx_map`, an array that maps each nonzero of the child Jacobian to the output entry it is added to. The axis convention and all other index decisions are handled at this point. The numeric pass is a single loop, the same for every axis and every representation. Indexing, transposition, and `kron` work the same way, with a copy instead of an addition.

Assembly at the problem level works in the same way. The solver receives one global CSR matrix, formed by concatenating the constraint Jacobians, whose layout is fixed during the symbolic pass; an evaluation only copies the values of each constraint root into this matrix at a fixed offset. Affine constraints are skipped entirely after the first evaluation, because their values cannot change until the parameters do.

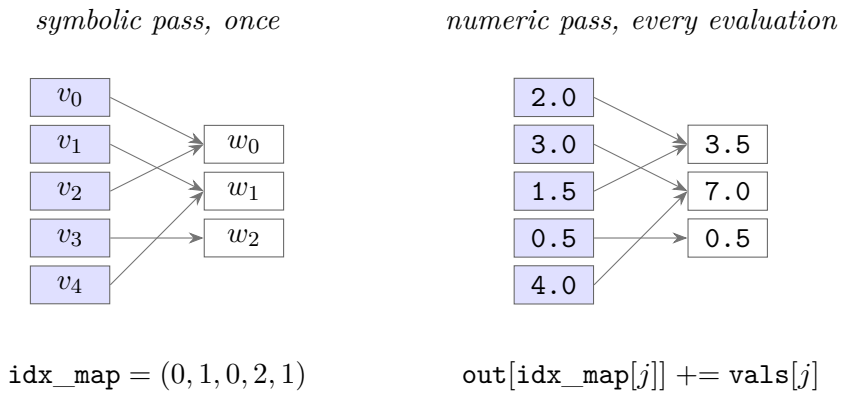


Figure 3.5 The two passes of the `sum` atom. During the symbolic pass, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The numeric passes of the other structural atoms have the same form.

A parameter update only involves the numeric pass: `update_params` writes the new values into the parameter leaves and sets a refresh flag on each affected subtree, and the next evaluation recomputes the values through the fixed maps and kernels. In comparison, without parametric compilation [26], `CVXPY` traverses the expression graph again, recomputes every sparsity pattern, and reallocates every buffer each time a parameter changes. The difference is measured in Chapter 4.

3.4 The API and the CVXPY interface

The engine is distributed as a C library with a thin Python binding called `sparsediffpy`, which exposes the node constructors. It is used in two places inside `CVXPY`. The nonlinear programming interface wraps the bindings in a class called `C_problem`, which is built from the expression trees of a problem. The canonicalization backend, which is the one measured in this thesis, is selected by passing `ignore_dpp=True`, and replaces the stuffed matrices of §2.1 with a cone program object that wraps the same `C_problem`. The code below is lightly condensed from the actual interface.

The lifecycle of a `C_problem` mirrors the symbolic and numeric passes:

```
prob = C_problem(problem)      # convert the tree, register parameters
prob.init_jacobian_coo()       # symbolic pass: patterns, maps, buffers
rows, cols = prob.get_jacobian_sparsity_coo() # structure, fetched once


prob.objective_forward(x)      # numeric pass, repeated per iterate
```

```
prob.constraint_forward(x)
grad = prob.gradient()
vals = prob.eval_jacobian_vals() # values aligned with (rows, cols)
```

The sparsity pattern is fetched once and does not change afterwards; the later calls return value arrays that are aligned with it.

The canonicalization backend extracts the problem data, an objective $\frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x}+\mathbf{q}^\top\mathbf{x}+d$ and constraints $\mathbf{A}\mathbf{x}+\mathbf{b}$ that must lie in a cone, by evaluating the oracles at $\mathbf{x}=0$. The forward passes give d and $\mathbf{b}$, the gradient gives $\mathbf{q}$, and the Jacobian and Hessian values fill the fixed patterns of $\mathbf{A}$ and $\mathbf{P}$. Parameter updates go through a single method:

```
def apply_parameters(self, id_to_value):
    theta = flatten_and_concatenate(id_to_value)
    if theta == self.cached_theta:    # same values: reuse everything
        return self.P, self.q, self.d, self.A, self.b
    self.c_problem.update_params(theta) # numeric pass only
    self.P, self.q, self.d, self.A, self.b = self.extract() # at x = 0
    self.cached_theta = theta
    return self.P, self.q, self.d, self.A, self.b
```

Repeated solves with unchanged parameter values return the cached matrices; an actual update costs one run of the numeric pass.

The caching follows the same pattern as in §3.3.2. The `C_problem` and its sparsity structures live as long as the problem family, the extracted matrices are kept until the parameter values change, and only numerical values are recomputed. After construction, nothing in the Python layer traverses an expression tree. Chapter 4 compares the engine with `CVXPY`'s canonicalization backends, implementations of the matrix-stuffing approach reviewed in Chapter 2.

The row-wise sparse product and the symbolic and numeric split are due to Gustavson [55]; see also [58]. Mixing dense blocks into sparse storage appears in several settings: supernodal and multifrontal solvers gather dense submatrices of a sparse factor and process them with level-3 BLAS [81, 82] (see also the survey [83]), and blocked sparse formats and autotuned kernels exploit small dense blocks found in the matrix [84, 85]. In these systems the blocks are discovered in the data; in the engine they are known from canonicalization.

CHAPTER 4 BENCHMARK RESULTS

This chapter presents benchmark results which compare the `diffengine` backend (the `CVXPY` implementation based on `SparseDiffEngine`) against `CVXPY`'s own canonicalization backends: the default `cvxcore` backend, the matrix-stuffing baseline of §2.1, and the two pure-Python backends, `SCIPY` and `COO`, that `CVXPY` ships as alternatives to it. The comparison is restricted to `CVXPY`'s own pipeline because that is the claim under test: `diffengine` is a drop-in replacement for the canonicalization backend, and the relevant question is whether it constructs the same problem data faster. All backends run on the 25 problems of `CVXPY`'s public benchmark suite (`https://github.com/cvxpy/benchmarks`). These convex optimization problems cover many fields, ranging from portfolio models, robust regression, and semidefinite programs to simple LPs and QPs that are close to standard form. The rest of the chapter first presents some technical details about the benchmark setup, followed by the comparison results which are paired with commentary and observations.

4.1 Benchmark Setup

Every benchmark measures the construction of the problem data (A, b, c, P); no solver is invoked. For both backends this corresponds to a call to `get_problem_data` with the target solver fixed to `Clarabel` [23]: the `CPP` (`cvxcore`) baseline runs on the default DPP path, while `diffengine` is selected by passing `ignore_dpp=True`. The alternative `SCIPY` and `COO` backends of §4.3 are selected through the `canon_backend` argument of the same call.

Every comparison reports up to two timings: a *single* solve, where the problem data are constructed once from scratch, and, on the subset of parametric problems, a *re-solve*, where the problem data are reconstructed after reassigning random parameter values on a cached problem.

All measurements run single-threaded (BLAS thread pools pinned to one thread) on a 2018 MacBook Pro with a 6-core Intel Core i7-8850H at 2.6 GHz and 16 GB of RAM. The software versions are:

- `Python` 3.14,
- `CVXPY` 1.10 (development version),
- `sparsediffpy` 0.6.1,

- `NumPy` 2.4 [39] and `SciPy` 1.17 [57].

Each cold compilation is measured once, in a fresh process per backend so that no cached compilation chain is reused; the re-solve columns are means of three re-evaluations. The `cvxcore` and `diffengine` timings were recorded in one session; the `SCIPY` and `COO` timings of §4.3 were recorded in a second session on the same machine and software versions, in which re-measured `cvxcore` times reproduced the first session within a few percent. Before any timing, `diffengine` is verified to reproduce the `cvxcore` problem data bit-identically ($\max|\Delta| = 0$) at full benchmark size. The scripts which run the benchmarks and all recorded timings are available at `https://github.com/SparseDifferentiation/Jacobian-accumulation` under the `canonicalization/` directory.

4.2 Single-solve compilation against `cvxcore`

A fresh problem is formed and canonicalized once from scratch (Table 4.1).

`diffengine` is faster on 24 of the 25 problems, with speedups between 1.2× and 124× and a geometric mean of 4.2×. The largest speedups occur where `cvxcore`'s per-expression tensor construction dominates the compilation time, as in the portfolio and regression models with many scalar constraints. UnconstrainedQP, a tiny problem wrapped in a large `kron` expression, is the extreme case at 124×. The speedups shrink on problems whose cost is a single large data copy, such as OptimalAdvertising and CVaR. Finally, the compilation of ConvexPlasticity, is dominated by `CVXPY`'s Python-side reductions rather than by either backend.

4.3 Single-solve compilation against the `SCIPY` and `COO` backends

Beyond the default C++ `cvxcore`, `CVXPY` ships two pure-Python canonicalization backends built on `scipy.sparse` [57]. The `SCIPY` backend represents each slice of the parameter tensor as a two-dimensional sparse matrix, while the more recent `COO` backend (introduced in 2025) keeps the whole tensor in coordinate format (§1.4.2) so that its operations scale with the number of stored nonzeros even when the parameters are large. Re-running the cold-compile experiment of §4.2 under these two backends answers whether the conclusions above depend on the choice of `cvxcore` as the baseline (Table 4.2). The `CPP` and `diffengine` columns repeat those of Table 4.1.

The sweep first shows that `cvxcore` is not uniformly the fastest of `CVXPY`'s own backends. The pure-Python `SCIPY` backend beats `CPP` on 18 of the 22 completed problems, usually by

Table 4.1 Cold compilation time across all 25 benchmarks, `diffengine` versus `cvxcore`. The ratio is `diffengine`/`CPP`; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

Benchmark	`CPP` (s)	`diffengine` (s)	Ratio
UnconstrainedQP	9.32	0.08	**0.01**
Cajas	19.66	1.06	0.05
FactorCovarianceModel	5.24	0.73	0.14
SVMWithL1Regularization	11.16	1.60	0.14
Yitzhaki	3.07	0.47	0.15
HuberRegression	10.97	1.61	0.15
LeastSquares	8.87	1.42	0.16
SimpleQP	10.49	1.69	0.16
SimpleScalarParametrizedLP	5.29	0.92	0.17
SimpleLP	30.07	5.29	0.18
SDPSegfault1132	59.75	12.47	0.21
SimpleFullyParametrizedLP	2.12	0.45	0.21
ParametrizedQP	10.19	2.25	0.22
TvInpainting	3.57	1.06	0.30
SlowPruning	6.88	2.15	0.31
SemidefiniteProgramming	5.38	1.93	0.36
ConeMatrixStuffing	10.34	3.76	0.36
QuantumHilbertMatrix	3.89	1.64	0.42
SmallMatrixStuffing	6.52	2.77	0.42
ParamSmallMatrixStuffing	0.48	0.21	0.43
ParamConeMatrixStuffing	0.31	0.14	0.45
Murray	5.79	3.43	0.59
CVaR	34.80	25.31	0.73
OptimalAdvertising	6.06	5.20	0.86
ConvexPlasticity	2.09	2.13	**1.02**
Geometric mean			**0.24**

Table 4.2 Cold compilation time under all three `CVXPY` canonicalization backends, compared with `diffengine`. The ratio is `diffengine` over the best of the three backends on each problem; values below 1 mean `diffengine` beats all of them. The final row is the geometric mean of the per-problem ratios over the 22 problems on which every backend completed.

Benchmark	`CPP` (s)	`SCIPY` (s)	`COO` (s)	`diffengine` (s)	Ratio
UnconstrainedQP	9.32	6.47	12.12	0.08	**0.01**
HuberRegression	10.97	10.07	10.61	1.61	0.16
LeastSquares	8.87	9.16	10.18	1.42	0.16
SimpleQP	10.49	10.17	11.23	1.69	0.17
Yitzhaki	3.07	2.80	3.11	0.47	0.17
FactorCovarianceModel	5.24	4.21	5.04	0.73	0.17
SVMWithL1Regularization	11.16	8.78	9.53	1.60	0.18
SimpleLP	30.07	28.92	27.23	5.29	0.19
SimpleScalarParametrizedLP	5.29	4.33	4.52	0.92	0.21
SlowPruning	6.88	6.85	7.58	2.15	0.31
TvInpainting	3.57	3.15	3.31	1.06	0.34
Cajas	19.66	2.92	3.17	1.06	0.36
ConeMatrixStuffing	10.34	9.44	16.49	3.76	0.40
SmallMatrixStuffing	6.52	7.20	10.28	2.77	0.42
QuantumHilbertMatrix	3.89	3.80	4.47	1.64	0.43
ParamSmallMatrixStuffing	0.48	27.96	0.42	0.21	0.49
ParamConeMatrixStuffing	0.31	7.45	0.27	0.14	0.52
SemidefiniteProgramming	5.38	2.31	3.53	1.93	0.84
Murray	5.79	3.73	3.87	3.43	0.92
CVaR	34.80	27.32	26.68	25.31	0.95
SDPSegfault1132	59.75	10.12	40.14	12.47	1.23
OptimalAdvertising	6.06	2.18	2.08	5.20	**2.50**
ConvexPlasticity	2.09	`ValueError`		2.13	—
SimpleFullyParametrizedLP	2.12	crash		0.45	—
ParametrizedQP	10.19	>1200 s		2.25	—
Geometric mean					**0.33**

a modest 1.0–1.3× but by 5.9× on SDPSegfault1132 and 6.7× on Cajas. Its weakness is the parameter-heavy problems, where building one tensor slice per parameter entry collapses: ParamSmallMatrixStuffing takes 28.0 s against `CPP`'s 0.48 s. The `COO` backend repairs exactly those rows (0.42 s and 0.27 s, on par with `CPP`), which is what its coordinate-format tensor is designed for, at the price of trailing `SCIPY` elsewhere (SDPSegfault1132, ConeMatrixStuffing). Neither Python backend completes the full suite: both raise a `ValueError` on ConvexPlasticity, the sweep lost SimpleFullyParametrizedLP to a crash of the benchmark process, and ParametrizedQP exceeded the 1200 s per-problem timeout.

Measured against an oracle that picks the fastest of the three backends on each problem, `diffengine` remains the fastest option on 18 of the 22 completed problems and is within 10 % of the best on two more (Murray, CVaR), with a geometric-mean ratio of 0.33, that is, 3.1× faster than the oracle. It is clearly beaten only on SDPSegfault1132 (1.2× slower than `SCIPY`) and OptimalAdvertising (2.5× slower than `COO`); as in §4.2, both are problems whose compilation is dominated by moving one large block of constant data into the problem matrices rather than by computing coefficients. The conclusions of §4.2 therefore do not hinge on `cvxcore` being the baseline.

4.4 Parametric compilation against `cvxcore`

On the subset of parametric problems, we report both the first solve and the subsequent re-solve time after reassigning random parameter values (Table 4.3).

`diffengine` is faster on the re-solve for six of the eight parametric problems, and on the first solve for all but ConvexPlasticity. Its re-solves are fast because the compiled program is cached, so only the parameter-dependent coefficient entries are re-evaluated. The DPP baseline instead multiplies its cached parameter tensor by the new parameter vector, which is hard to beat when that tensor is small and dense. This explains the two rows where DPP remains around 2× faster: a scalar parameter whose effect is spread across many coefficients (SVM's regularization weight, FactorCovariance's risk aversion) is the ideal case for the tensor product.

Table 4.3 Parametric compilation, `diffengine` versus `cvxcore`: first solve and warm re-solve. The ratio is `diffengine`/`CPP` on the re-solve; values below 1 favor `diffengine`. The final row is the geometric mean of the per-problem ratios.

		First solve (s)		Re-solve (s)		
Benchmark	#par	`CPP`	`diffengine`	`CPP`	`diffengine`	Ratio
ParamConeMatrixStuffing	3	0.31	0.14	0.0030	0.0010	**0.33**
ParamSmallMatrixStuffing	3	0.48	0.21	0.0048	0.0017	0.35
SimpleFullyParametrizedLP	1	2.12	0.45	0.230	0.123	0.54
ParametrizedQP	2	10.19	2.25	1.361	0.959	0.70
SimpleScalarParametrizedLP	1	5.29	0.92	0.330	0.251	0.76
ConvexPlasticity	3	2.09	2.13	0.668	0.666	1.00
FactorCovarianceModel	2	5.24	0.73	0.144	0.277	1.92
SVMWithL1Regularization	1	11.16	1.60	0.263	0.530	**2.01**
Geometric mean						**0.78**

CHAPTER 5 CONCLUSION

5.1 Summary of Works

5.2 Limitations

5.3 Future Research

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," Alfred P. Sloan School of Management, Massachusetts Institute of Technology, Cambridge, MA, Tech. Rep., 1978.

[2] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[3] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[4] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[5] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[6] J. Bisschop, *AIMMS Optimization Modeling*. Haarlem, The Netherlands: Paragon Decision Technology, 2006.

[7] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[8] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[9] M. L. Bynum *et al.*, *Pyomo—Optimization Modeling in Python*. Springer, 2021.

[10] W. E. Hart *et al.*, "Pyomo: modeling and solving mathematical programs in python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[13] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[14] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[15] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[16] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[17] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[18] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[19] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[20] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[21] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[22] D. Ge, Q. Huangfu, Z. Wang, J. Wu, and Y. Ye, "Cardinal Optimizer (COPT) user guide," *arXiv preprint arXiv:2208.14314*, 2022.

[23] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.

[24] A. Ben-Tal and A. Nemirovski, *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*. SIAM, 2001.

[25] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[26] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[27] W. Zhang, P. Nobel, A. Jeendgar, R. Murray, P. Schiele, and S. Diamond, "CVXPY 1.9: Recent advances in optimization modeling software," 2026. [Online]. Available: https://arxiv.org/abs/2606.14891

[28] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *IMA Journal of Applied Mathematics*, vol. 13, no. 1, pp. 117–119, 1974. [Online]. Available: https://doi.org/10.1093/imamat/13.1.117

[29] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[30] D. M. Gay, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[31] D. M. Gay, "Hooking your solver to AMPL," Bell Laboratories, Tech. Rep. 97-4-06, 1997, reference for the AMPL Solver Library (ASL).

[32] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[33] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

[34] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2025. [Online]. Available: https://arxiv.org/abs/2606.02896

[35] E. M. B. Smith, "On the optimal design of continuous processes," Ph.D. dissertation, Imperial College London (University of London), London, 1996, ph.D. dissertation.

[36] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[37] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[38] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[39] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[40] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[41] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[42] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[43] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[44] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario, 2026. [Online]. Available: https://www.maplesoft.com

[45] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[46] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[47] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[48] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for Fortran usage," *ACM Transactions on Mathematical Software*, vol. 5, no. 3, pp. 308–323, 1979.

[49] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Transactions on Mathematical Software*, vol. 16, no. 1, pp. 1–17, 1990.

[50] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*, 3rd ed. Philadelphia, PA: SIAM, 1999.

[51] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[52] H. M. Markowitz, "The elimination form of the inverse and its application to linear programming," *Management Science*, vol. 3, no. 3, pp. 255–269, 1957.

[53] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[54] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[55] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software*, vol. 4, no. 3, pp. 250–269, 1978.

[56] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[57] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[58] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Advances in Computational Mathematics*, vol. 1, no. 1, pp. 127–137, 1993.

[59] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," 2010. [Online]. Available: http://eigen.tuxfamily.org

[60] I. S. Duff, M. A. Heroux, and R. Pozo, "An overview of the sparse basic linear algebra subprograms: The new standard from the BLAS technical forum," *ACM Transactions on Mathematical Software*, vol. 28, no. 2, pp. 239–267, 2002.

[61] J. R. Magnus and H. Neudecker, *Matrix Differential Calculus with Applications in Statistics and Econometrics*, 3rd ed., ser. Wiley Series in Probability and Statistics. John Wiley & Sons, 2019.

[62] P. S. Dwyer and M. S. Macphail, "Symbolic matrix derivatives," *The Annals of Mathematical Statistics*, vol. 19, no. 4, pp. 517–534, 1948.

[63] The CVXPY developers, "CVXPY source code repository," https://github.com/cvxpy/cvxpy, 2026.

[64] The CVXPY developers, "CVXPY documentation," https://www.cvxpy.org/, 2026.

[65] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[66] M. Blondel and V. Roulet, "The elements of differentiable programming," 2024, arXiv:2403.14606. [Online]. Available: https://arxiv.org/abs/2403.14606

[67] A. Hill and G. Dalle, "Sparser, better, faster, stronger: Sparsity detection for efficient automatic differentiation," *arXiv preprint arXiv:2501.17737*, 2025. [Online]. Available: https://arxiv.org/abs/2501.17737

[68] A. Griewank and S. Reese, "On the calculation of Jacobian matrices by the Markowitz rule," in *Automatic Differentiation of Algorithms: Theory, Implementation, and Application*, A. Griewank and G. F. Corliss, Eds. Philadelphia, PA: SIAM, 1991, pp. 126–135.

[69] U. Naumann, "Optimal Jacobian accumulation is NP-complete," *Mathematical Programming*, vol. 112, no. 2, pp. 427–441, 2008. [Online]. Available: https://doi.org/10.1007/s10107-006-0042-z

[70] A. Griewank, D. Juedes, and J. Utke, "Algorithm 755: ADOL-C: a package for the automatic differentiation of algorithms written in C/C++," *ACM Transactions on Mathematical Software*, vol. 22, no. 2, pp. 131–167, 1996. [Online]. Available: https://doi.org/10.1145/229473.229474

[71] B. M. Bell, "CppAD: A package for C++ algorithmic differentiation," 2023, computational Infrastructure for Operations Research. [Online]. Available: https://github.com/coin-or/CppAD

[72] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[73] T. F. Coleman and J. J. Moré, "Estimation of sparse Jacobian matrices and graph coloring problems," *SIAM Journal on Numerical Analysis*, vol. 20, no. 1, pp. 187–209, 1983. [Online]. Available: https://doi.org/10.1137/0720013

[74] M. J. D. Powell and P. L. Toint, "On the estimation of sparse Hessian matrices," *SIAM Journal on Numerical Analysis*, vol. 16, no. 6, pp. 1060–1074, 1979. [Online]. Available: https://doi.org/10.1137/0716078

[75] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J. on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[76] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[77] CasADi developers, "CasADi user guide," https://web.casadi.org/docs/, 2025.

[78] D. M. Gay, "Automatic differentiation of nonlinear AMPL models," AT&T Bell Laboratories, Tech. Rep. Numerical Analysis Manuscript 91-05, 1991, reverse mode AD on AMPL expression graphs.

[79] D. M. Gay, "More AD of nonlinear AMPL models: Computing hessian information and exploiting partial separability," in *Computational Differentiation: Techniques, Applications, and Tools*. SIAM, 1996, sparse Hessians via partial separability, no graph coloring.

[80] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[81] I. S. Duff and J. K. Reid, "The multifrontal solution of indefinite sparse symmetric linear equations," *ACM Transactions on Mathematical Software*, vol. 9, no. 3, pp. 302–325, 1983.

[82] J. W. Demmel, S. C. Eisenstat, J. R. Gilbert, X. S. Li, and J. W. H. Liu, "A supernodal approach to sparse partial pivoting," *SIAM Journal on Matrix Analysis and Applications*, vol. 20, no. 3, pp. 720–755, 1999.

[83] J. W. H. Liu, "The multifrontal method for sparse matrix solution: Theory and practice," *SIAM Review*, vol. 34, no. 1, pp. 82–109, 1992.

[84] E.-J. Im, K. Yelick, and R. Vuduc, "Sparsity: Optimization framework for sparse matrix kernels," *International Journal of High Performance Computing Applications*, vol. 18, no. 1, pp. 135–158, 2004.

[85] R. Vuduc, J. W. Demmel, and K. A. Yelick, "OSKI: A library of automatically tuned sparse matrix kernels," *Journal of Physics: Conference Series*, vol. 16, pp. 521–530, 2005.